SNOWFLAKE PRACTICAL DOCUMENTATION

Sales Data Loading, Cleaning, Transformation and SQL Optimization

STEP 1: CREATE DATABASE AND SCHEMA

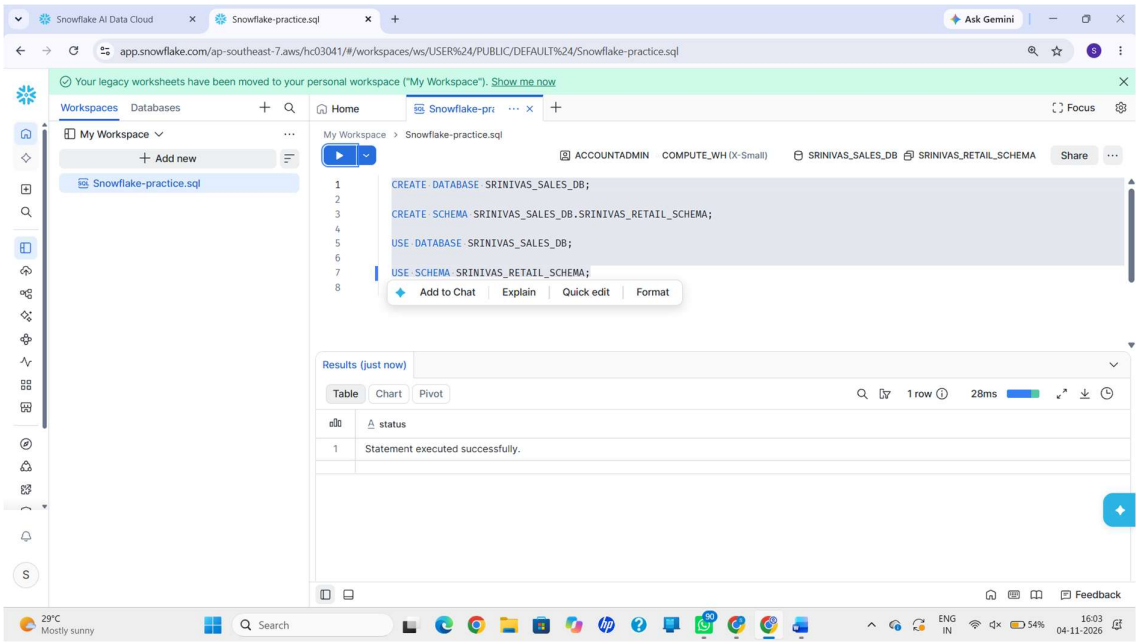
First, create a database and schema to organize the sales project.

```
CREATE DATABASE SRINIVAS_SALES_DB;
CREATE SCHEMA SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA;
USE DATABASE SRINIVAS_SALES_DB;
USE SCHEMA SRINIVAS_RETAIL_SCHEMA;
```

Explanation:

- DATABASE: Stores the project-level database objects.
- SCHEMA: Organizes related tables, stages and file formats.
- USE DATABASE: Selects the database.
- USE SCHEMA: Selects the schema.

Screenshot 1: Database and Schema Creation



STEP 2: CREATE WAREHOUSE

A warehouse provides the compute resources required to execute SQL queries.

```
CREATE WAREHOUSE IF NOT EXISTS SRINIVAS_WH
```

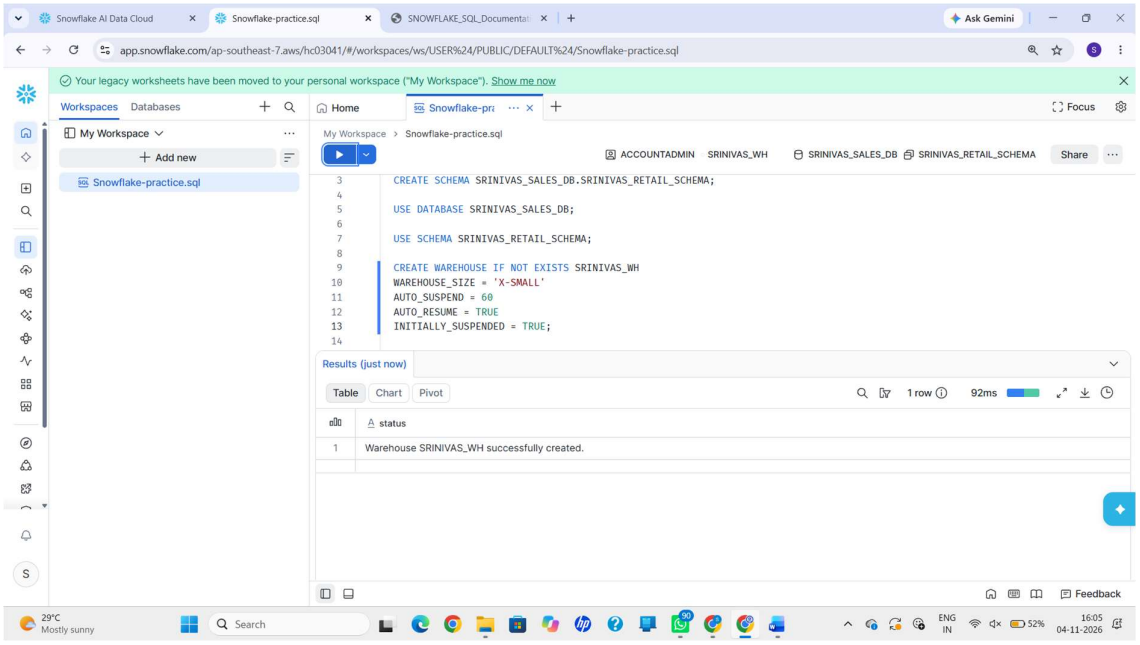
```
WAREHOUSE_SIZE = 'X-SMALL'
AUTO_SUSPEND = 60
AUTO_RESUME = TRUE
INITIALLY_SUSPENDED = TRUE;

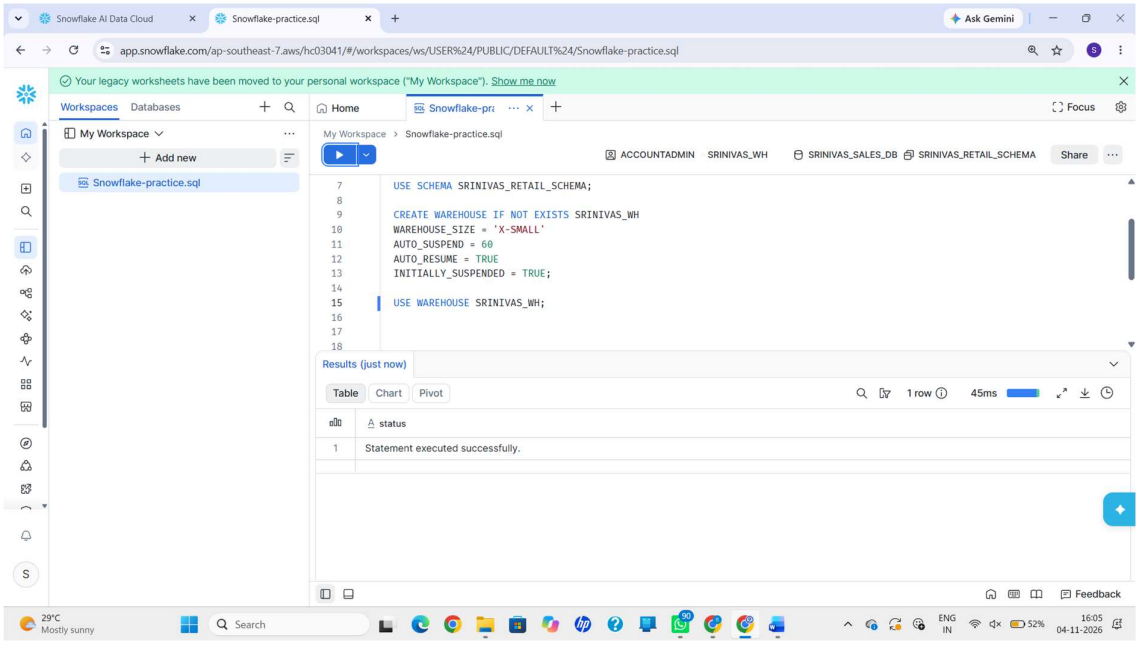
USE WAREHOUSE SRINIVAS_WH;
```

Explanation:

- X-SMALL: Specifies the warehouse size.
- AUTO_SUSPEND: Automatically suspends the warehouse after 60 seconds of inactivity.
- AUTO_RESUME: Automatically resumes the warehouse when a query is submitted.
- INITIALLY_SUSPENDED: Creates the warehouse in suspended state.

Screenshot 2: Warehouse Creation





STEP 3: CREATE FILE FORMAT

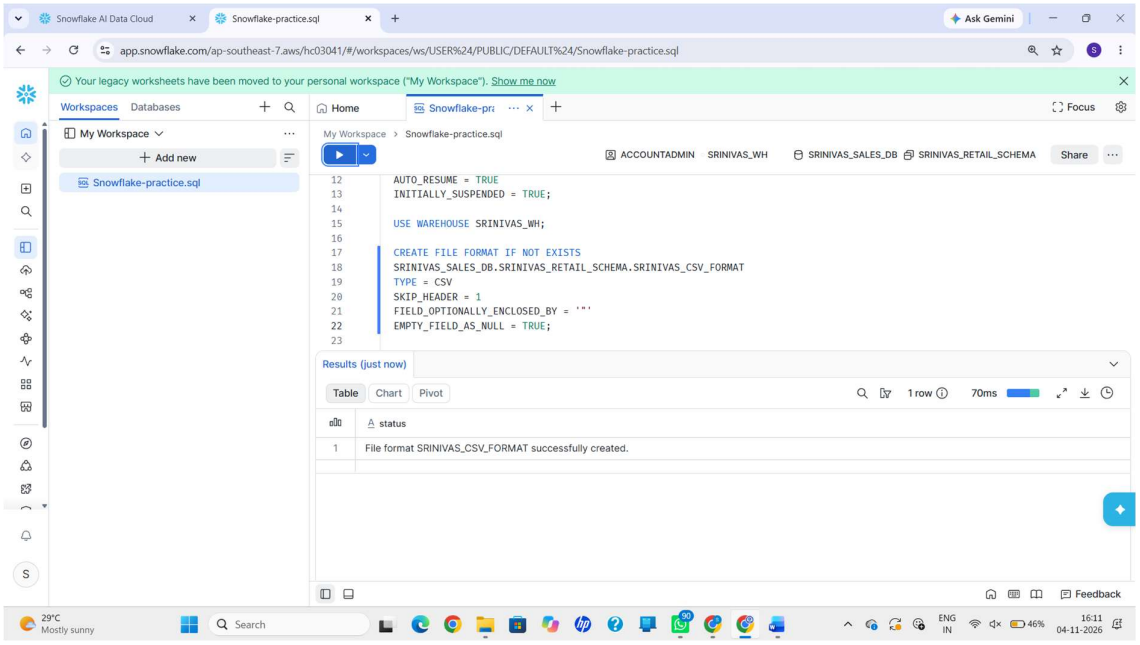
A file format specifies how Snowflake should interpret the CSV files.

```
CREATE FILE FORMAT IF NOT EXISTS
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CSV_FORMAT
TYPE = CSV
SKIP_HEADER = 1
FIELD_OPTIONALLY_ENCLOSED_BY = '"'
EMPTY_FIELD_AS_NULL = TRUE;
```

Explanation:

- TYPE = CSV: Specifies the file type.
- SKIP_HEADER = 1: Skips the header row.
- FIELD_OPTIONALLY_ENCLOSED_BY: Handles values enclosed in double quotes.
- EMPTY_FIELD_AS_NULL: Converts empty fields into NULL.

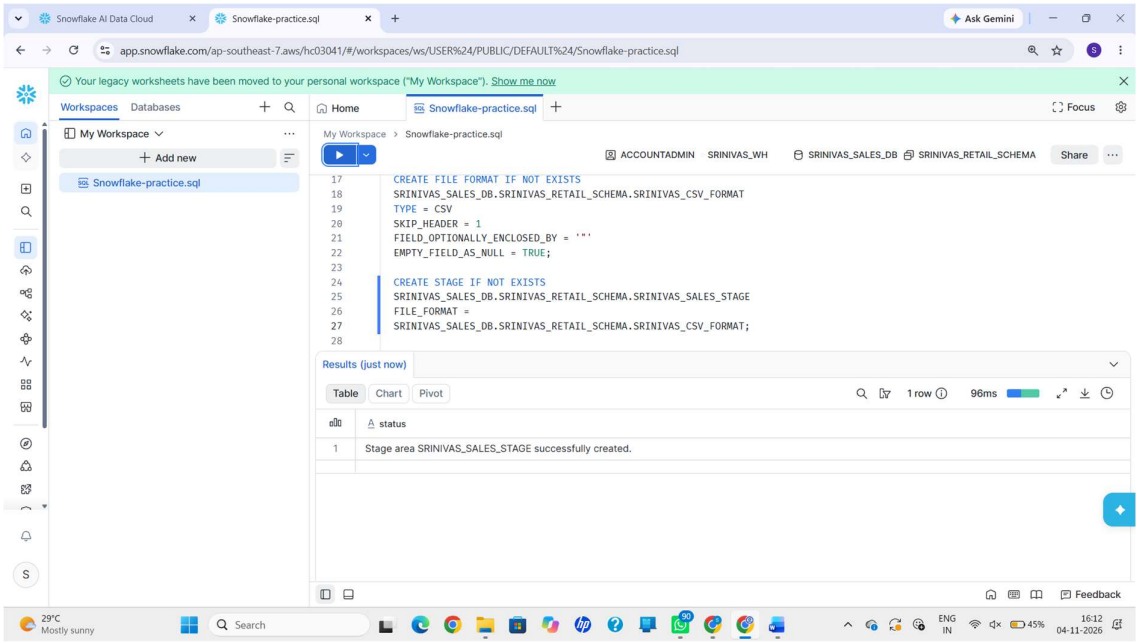
Screenshot 3: File Format Creation



STEP 4: CREATE INTERNAL STAGE

An internal stage is used to temporarily store files before loading them into Snowflake tables.

```
CREATE STAGE IF NOT EXISTS
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_SALES_STAGE
FILE_FORMAT =
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CSV_FORMAT;
```

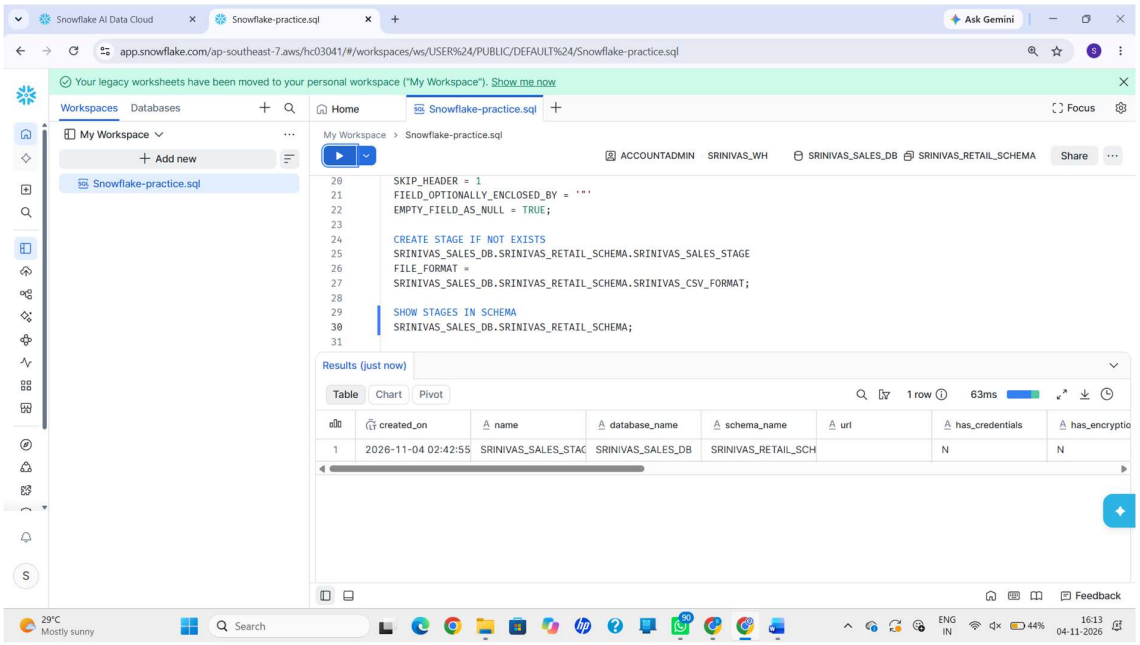


Verify the stage:

SHOW STAGES IN SCHEMA

SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA;

Screenshot 4: Internal Stage Creation



STEP 5: UPLOAD CSV FILES TO STAGE

Upload the following files into the internal stage using the Snowflake Snowsight interface:

1. SRINIVAS_CUSTOMERS.csv
2. SRINIVAS_SALES_RAW.csv

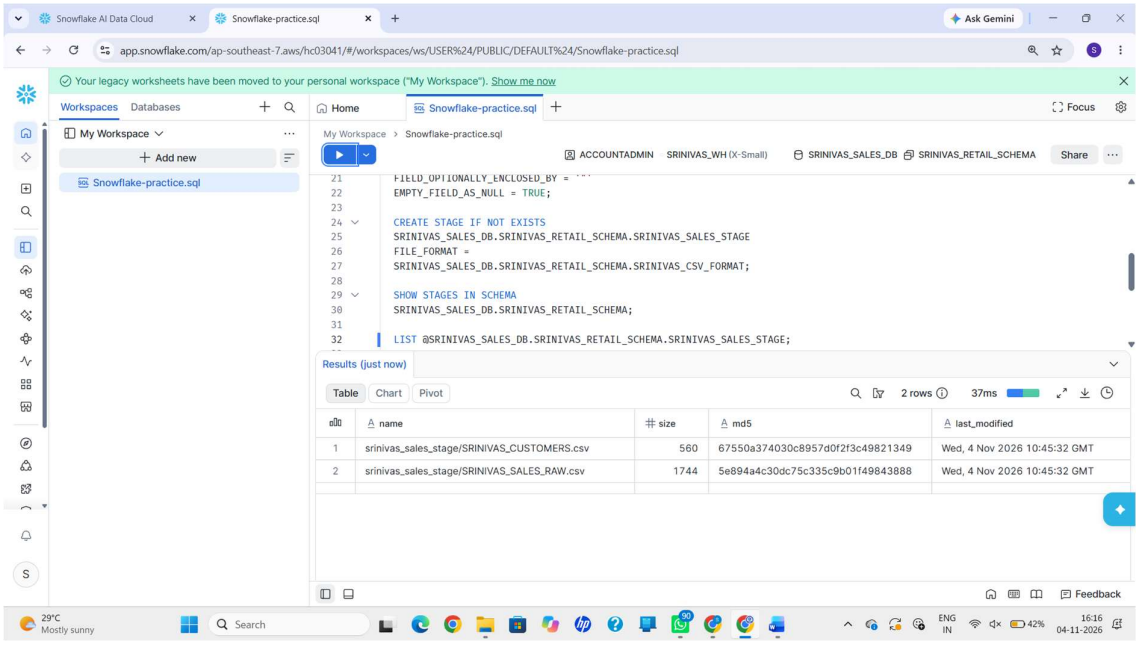
After uploading, execute:

```
LIST
@SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_SALES_STAGE;
```

Explanation:

The LIST command displays the files available inside the stage.

Screenshot 5: CSV Files Uploaded to Internal Stage



STEP 6: CREATE CUSTOMERS TABLE

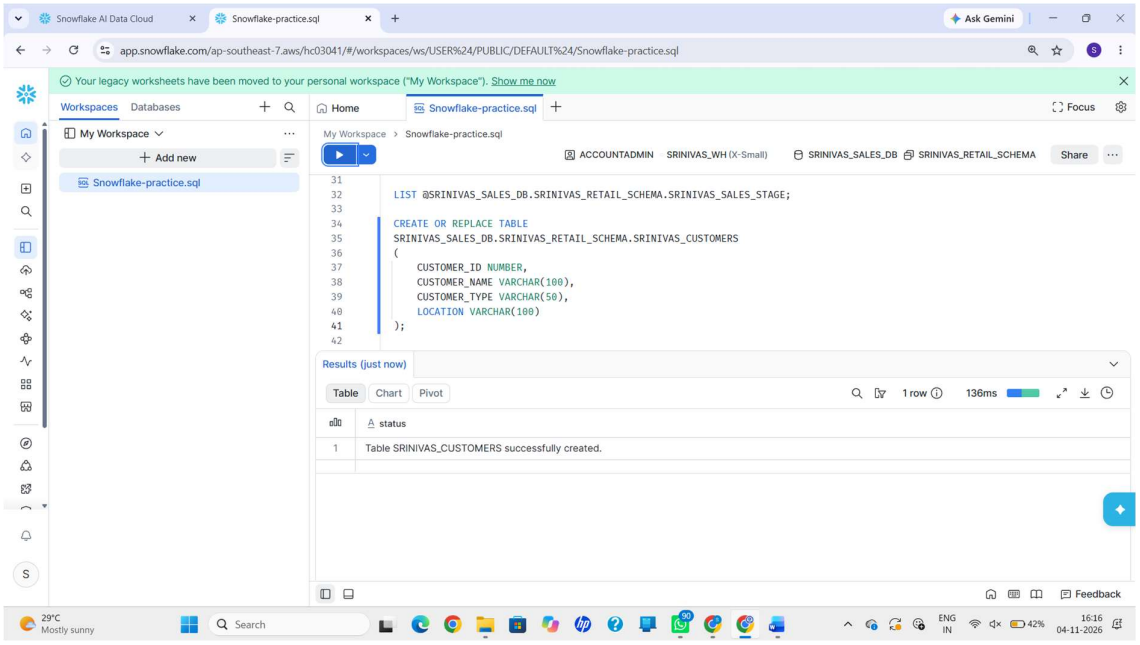
Create the first table to store clean customer information.

```
CREATE OR REPLACE TABLE
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CUSTOMERS
(
   CUSTOMER_ID NUMBER,
   CUSTOMER_NAME VARCHAR(100),
   CUSTOMER_TYPE VARCHAR(50),
   LOCATION VARCHAR(100)
);
```

Explanation:

The table contains customer identifiers, names, customer types and locations.

Screenshot 6: Customers Table Creation



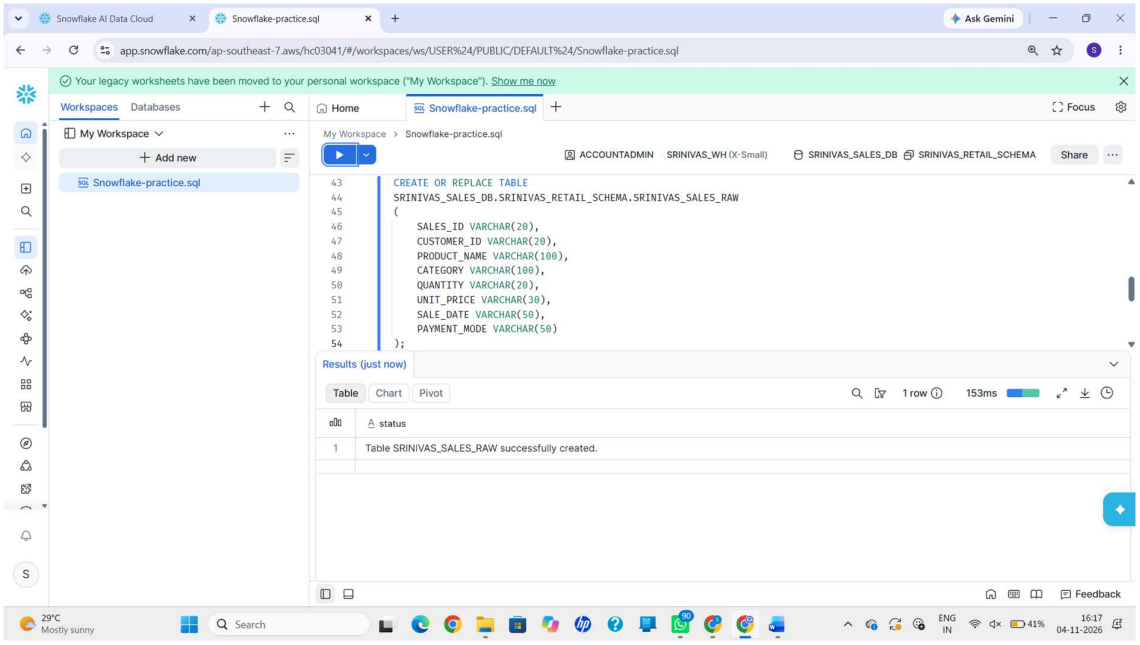
STEP 7: CREATE RAW SALES TABLE

Create the second table to store the sales data.

Initially, all columns are defined using VARCHAR because the source dataset contains inconsistent date and numeric formats.

```
CREATE OR REPLACE TABLE
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_SALES_RAW
(
   SALES_ID VARCHAR(20),
   CUSTOMER_ID VARCHAR(20),
   PRODUCT_NAME VARCHAR(100),
   CATEGORY VARCHAR(100),
   QUANTITY VARCHAR(20),
   UNIT_PRICE VARCHAR(30),
   SALE_DATE VARCHAR(50),
   PAYMENT_MODE VARCHAR(50)
);
```

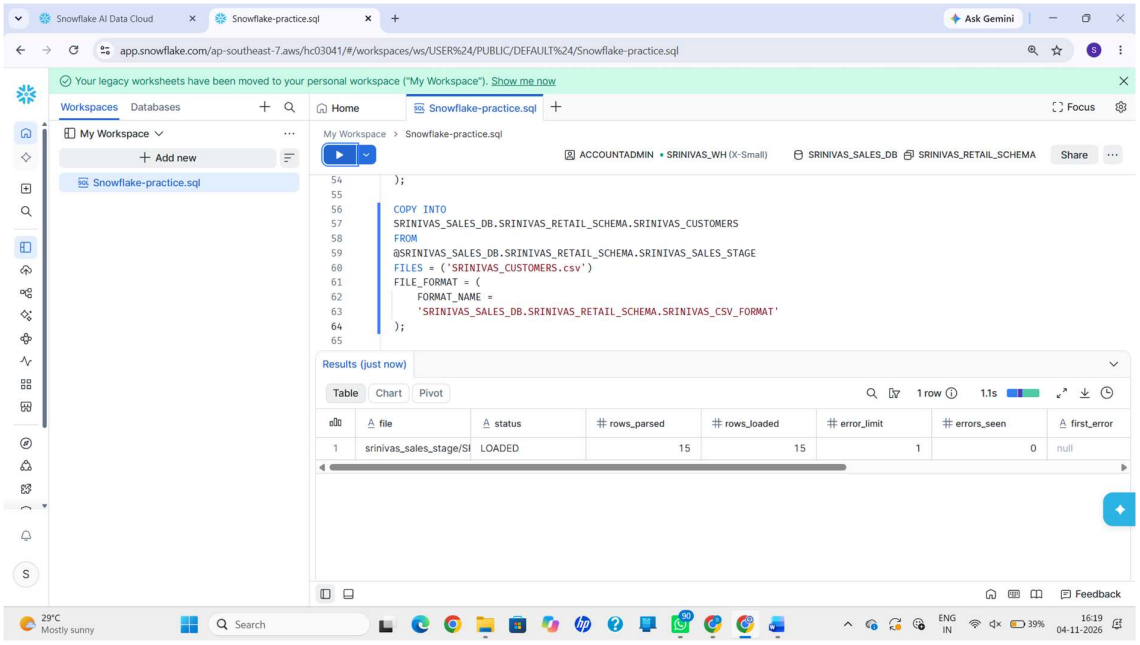
Screenshot 7: Raw Sales Table Creation



STEP 8: LOAD DATA USING COPY INTO

Load the customer CSV file.

```
COPY INTO
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CUSTOMERS
FROM
@SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_SALES_STAGE
FILES = ('SRINIVAS_CUSTOMERS.csv')
FILE_FORMAT = (
   FORMAT_NAME =
   'SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CSV_FORMAT'
);
```



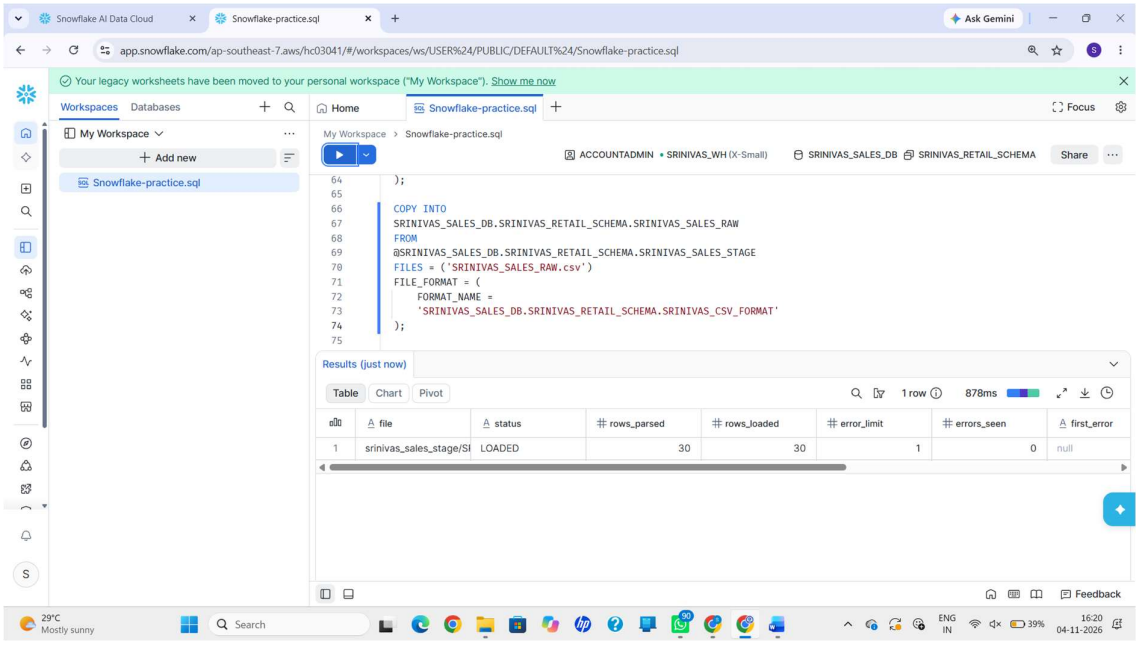
Load the sales CSV file.

```
COPY INTO
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_SALES_RAW
FROM
@SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_SALES_STAGE
FILES = ('SRINIVAS_SALES_RAW.csv')
FILE_FORMAT = (
   FORMAT_NAME =
   'SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CSV_FORMAT'
);
```

Explanation:

COPY INTO is used to load data from staged files into Snowflake tables.

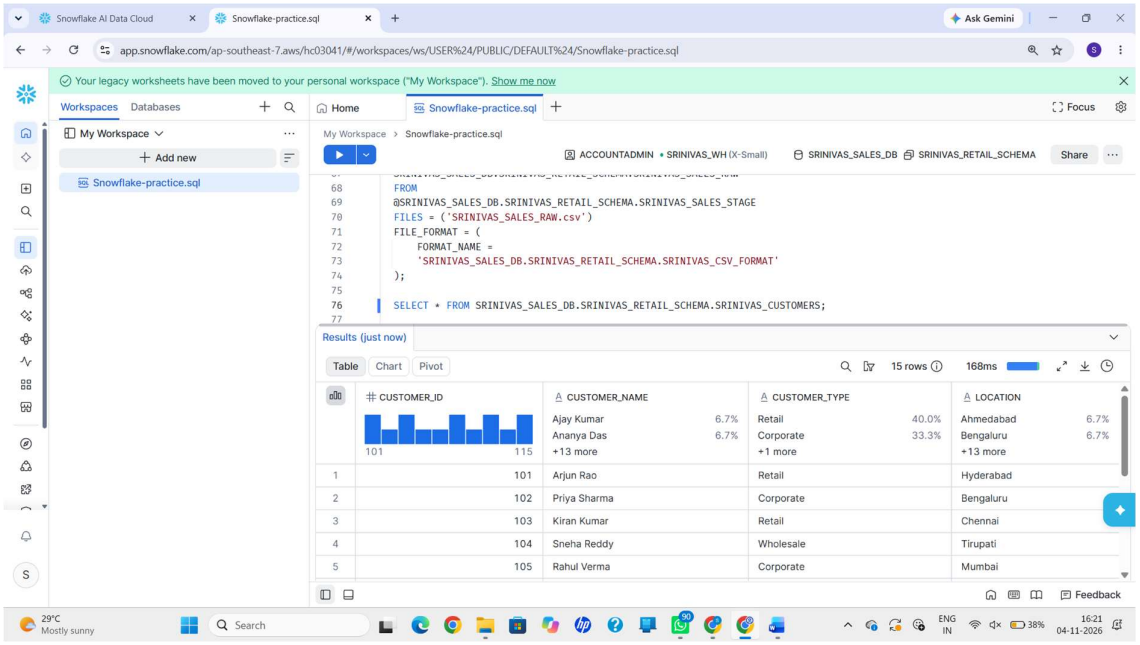
Screenshot 8: COPY INTO Execution Results



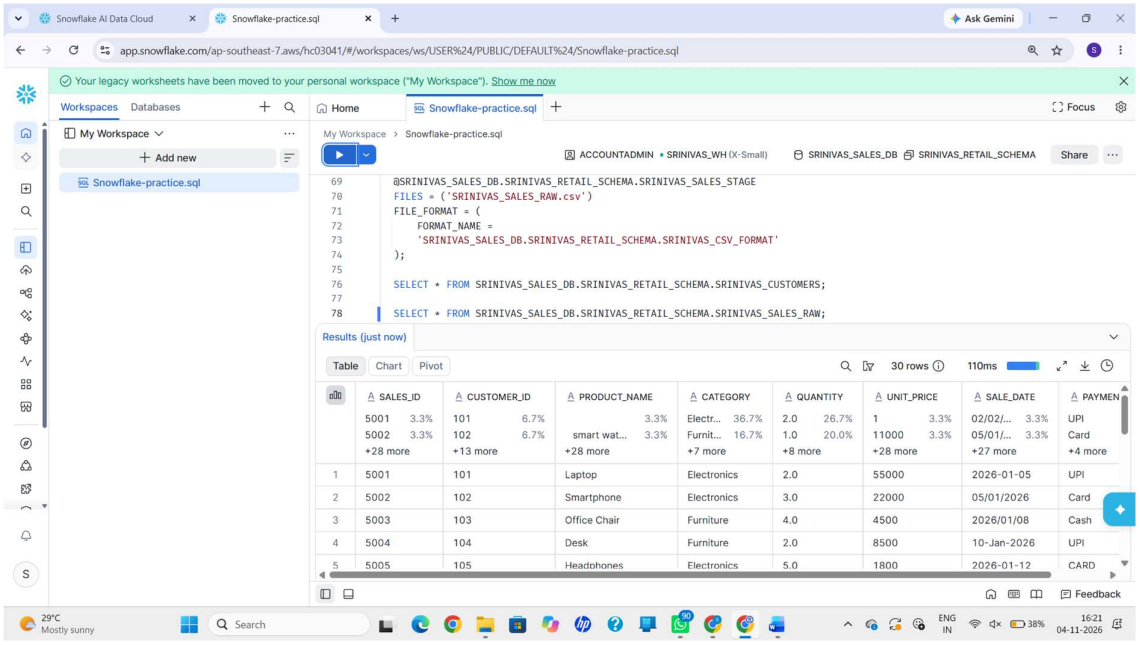
STEP 9: VERIFY LOADED DATA

Display all records from both tables.

```
SELECT * FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CUSTOMERS;
```

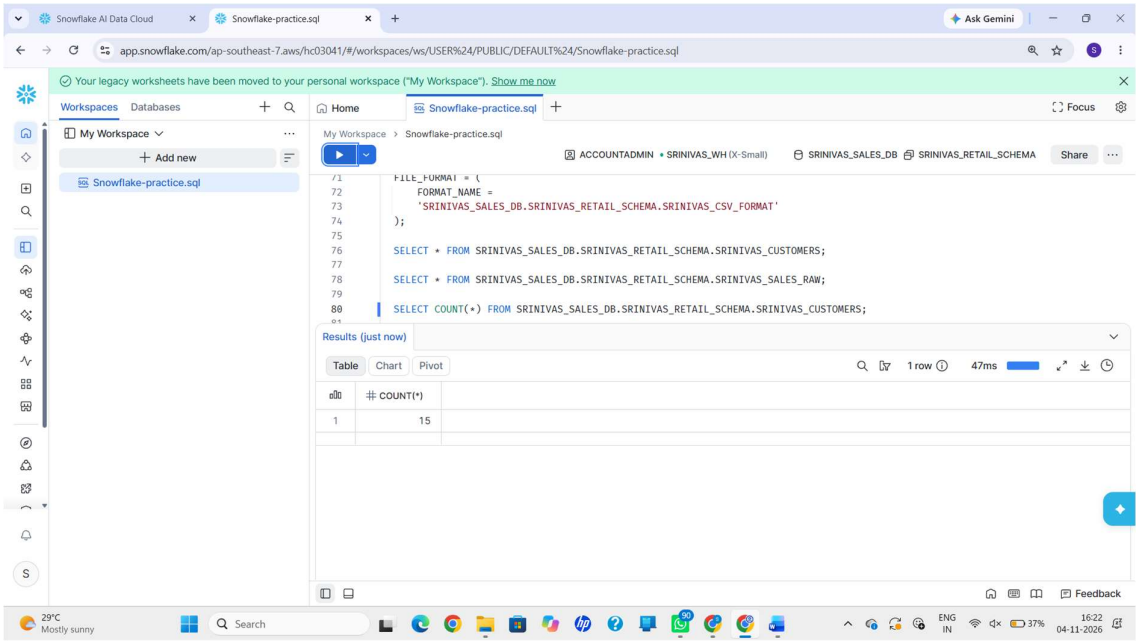



```
SELECT * FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_SALES_RAW;
```



Check the number of records.

```
SELECT COUNT(*) FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CUSTOMERS;
```

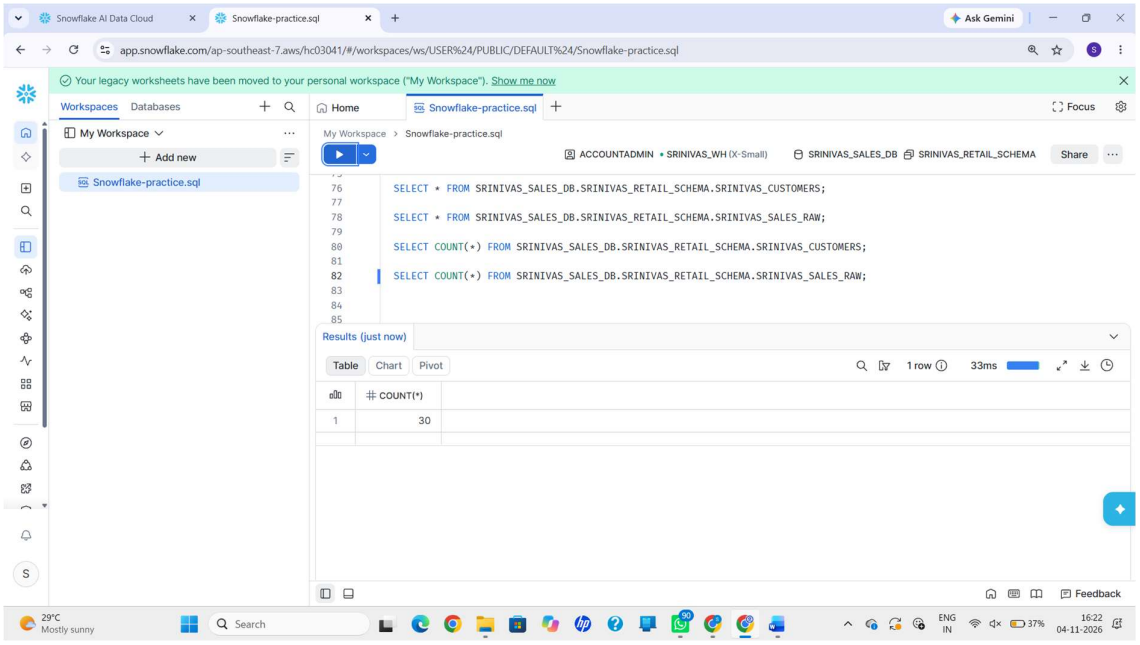



```
SELECT COUNT(*) FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_SALES_RAW;
```

Expected Results:

- Customers: 15 records.
- Sales Raw: 30 records.

Screenshot 9: Loaded Data and Record Counts



6. DATA CLEANING AND TRANSFORMATION

STEP 10: IDENTIFY DATA-QUALITY ISSUES

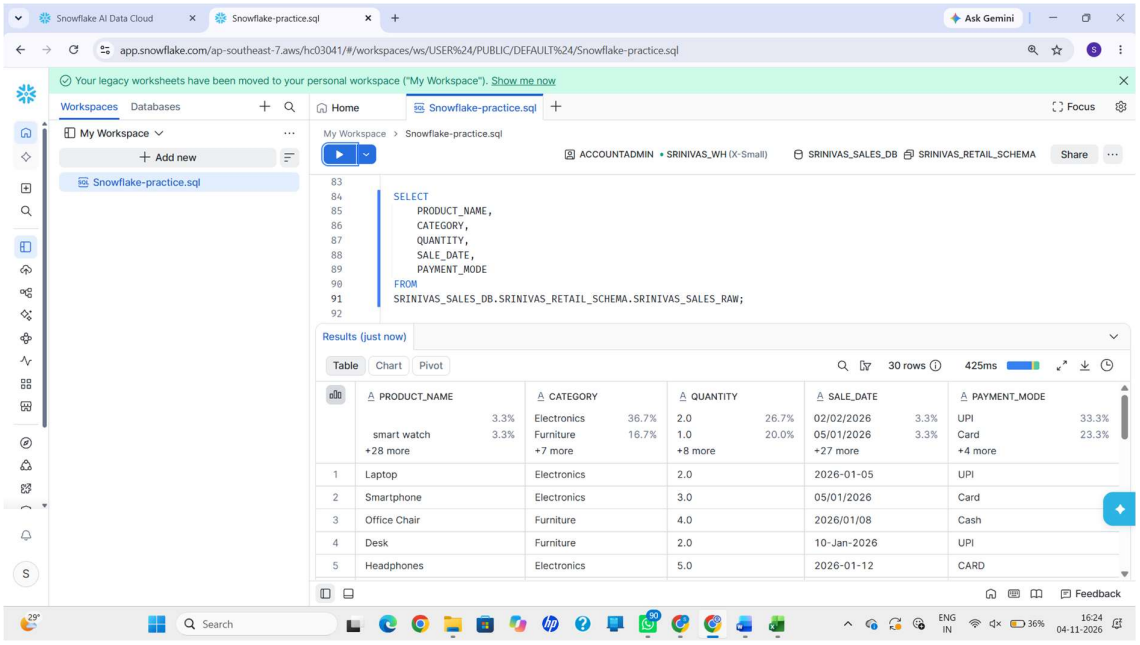
Inspect the raw sales data.

```
SELECT
   PRODUCT_NAME,
   CATEGORY,
   QUANTITY,
   SALE_DATE,
   PAYMENT_MODE
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_SALES_RAW;
```

The dataset contains:

- Extra spaces in product names.
- Inconsistent category formatting.
- Missing quantity values.
- Missing category values.
- Different date formats.
- Missing payment modes.

Screenshot 10: Identification of Dirty Data



STEP 11: UNDERSTAND DATA-CLEANING FUNCTIONS

A. TRIM()

Removes unwanted spaces from the beginning and end of a string.

```
SELECT TRIM('   LAPTOP   ');
```

B. UPPER()

Converts lowercase characters into uppercase.

```
SELECT UPPER('electronics');
```

C. COALESCE()

Returns the first non-NULL value from the supplied expressions.

```
SELECT COALESCE(NULL, 'UNKNOWN');
SELECT COALESCE(NULL,NULL,‘HI’, 'UNKNOWN'); // gives Hi
SELECT COALESCE(‘’, 'UNKNOWN');  // gives ‘’
```

D. NULLIF()

Returns NULL when two expressions are equal.

```
SELECT NULLIF('', '');
```

E. TRY_TO_NUMBER()

Converts text into a number without raising an error for invalid input.

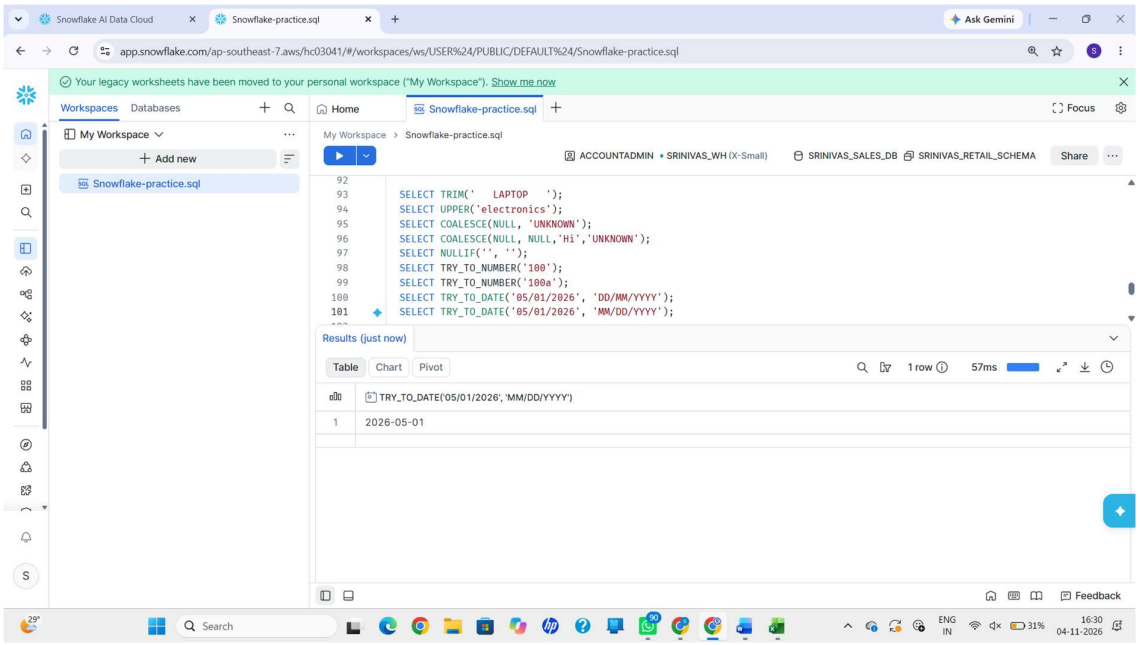
```
SELECT TRY_TO_NUMBER('100');
```

F. TRY_TO_DATE()

Converts text into a date using a specified format.

```
SELECT TRY_TO_DATE('05/01/2026', 'DD/MM/YYYY');
```

Screenshot 11: Data-Cleaning Function Execution



STEP 12: CREATE CLEAN SALES TABLE

Perform all required transformations and create a new clean table.

```
CREATE OR REPLACE TABLE
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE AS
SELECT
   TRY_TO_NUMBER(SALES_ID) AS SALES_ID,
   TRY_TO_NUMBER(CUSTOMER_ID) AS CUSTOMER_ID,
   COALESCE( NULLIF(UPPER(TRIM(PRODUCT_NAME)), ''),'UNKNOWN PRODUCT' ) AS PRODUCT_NAME,
   COALESCE( NULLIF(UPPER(TRIM(CATEGORY)), ''),'UNCATEGORIZED') AS CATEGORY,
   COALESCE( TRY_TO_NUMBER(QUANTITY), 0) AS QUANTITY,
   COALESCE( TRY_TO_DECIMAL(UNIT_PRICE, 12, 2),  0) AS UNIT_PRICE,
   CASE
      WHEN TRY_TO_DATE(TRIM(SALE_DATE), 'YYYY-MM-DD') IS NOT NULL
      THEN TRY_TO_DATE(TRIM(SALE_DATE), 'YYYY-MM-DD')
```

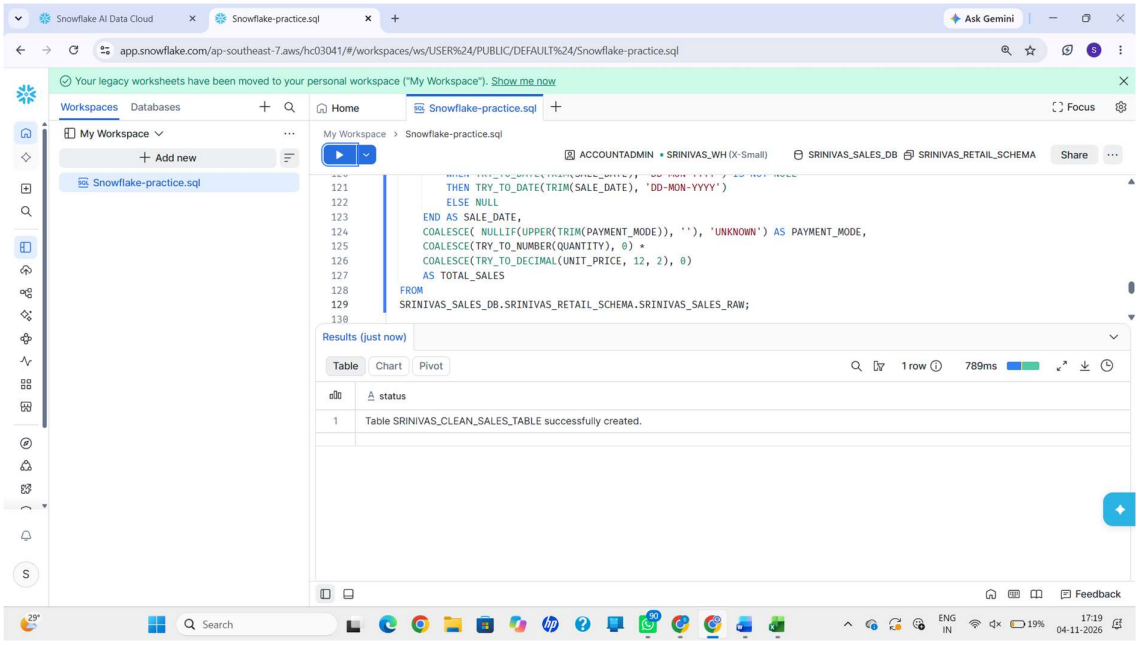
```
    WHEN TRY_TO_DATE(TRIM(SALE_DATE), 'DD/MM/YYYY') IS NOT NULL
    THEN TRY_TO_DATE(TRIM(SALE_DATE), 'DD/MM/YYYY')
    WHEN TRY_TO_DATE(TRIM(SALE_DATE), 'YYYY/MM/DD') IS NOT NULL
    THEN TRY_TO_DATE(TRIM(SALE_DATE), 'YYYY/MM/DD')
    WHEN TRY_TO_DATE(TRIM(SALE_DATE), 'DD-MON-YYYY') IS NOT NULL
    THEN TRY_TO_DATE(TRIM(SALE_DATE), 'DD-MON-YYYY')
    ELSE NULL
  END AS SALE_DATE,
  COALESCE( NULLIF(UPPER(TRIM(PAYMENT_MODE)), ''), 'UNKNOWN') AS
PAYMENT_MODE,
  COALESCE(TRY_TO_NUMBER(QUANTITY), 0) *
  COALESCE(TRY_TO_DECIMAL(UNIT_PRICE, 12, 2), 0)
  AS TOTAL_SALES
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_SALES_RAW;
```

Transformations performed:

Operation	Description
TRIM	Removes unwanted spaces
UPPER	Standardizes text to uppercase
COALESCE	Replaces NULL values with defaults
NULLIF	Converts empty strings into NULL
TRY_TO_NUMBER	Converts numeric text into numbers
TRY_TO_DATE	Converts different date formats
CASE	Handles multiple date formats
TOTAL_SALES	Calculates quantity multiplied by unit price

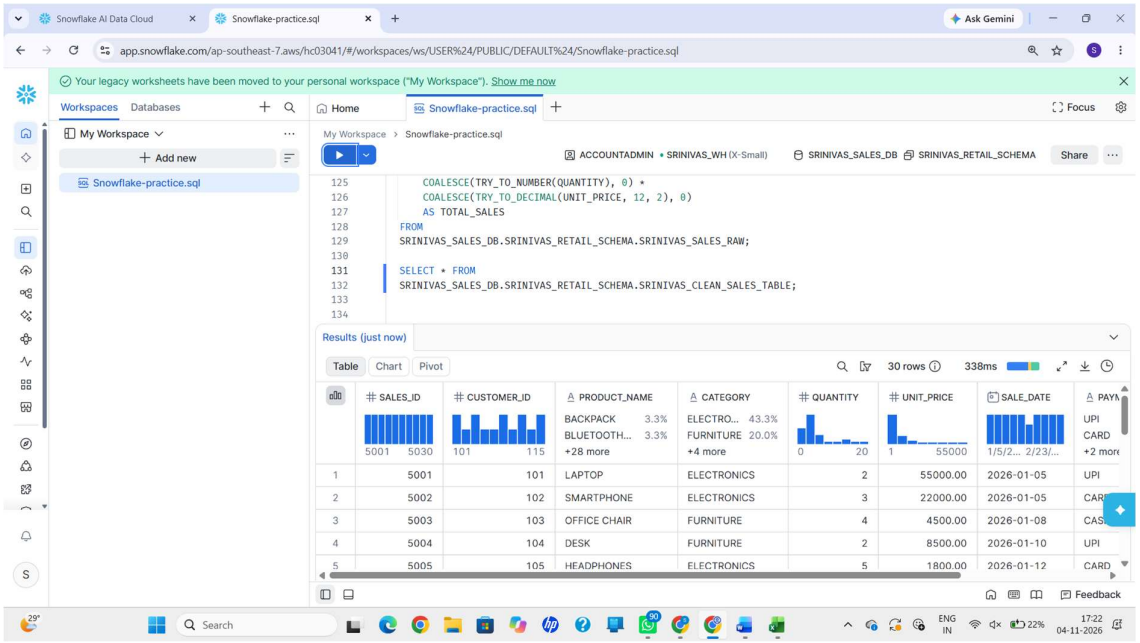
Note: Missing quantity and price are replaced with zero for example.

Screenshot 12: Clean Table Creation



STEP 13: DISPLAY CLEAN TABLE

```
SELECT * FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE;
```

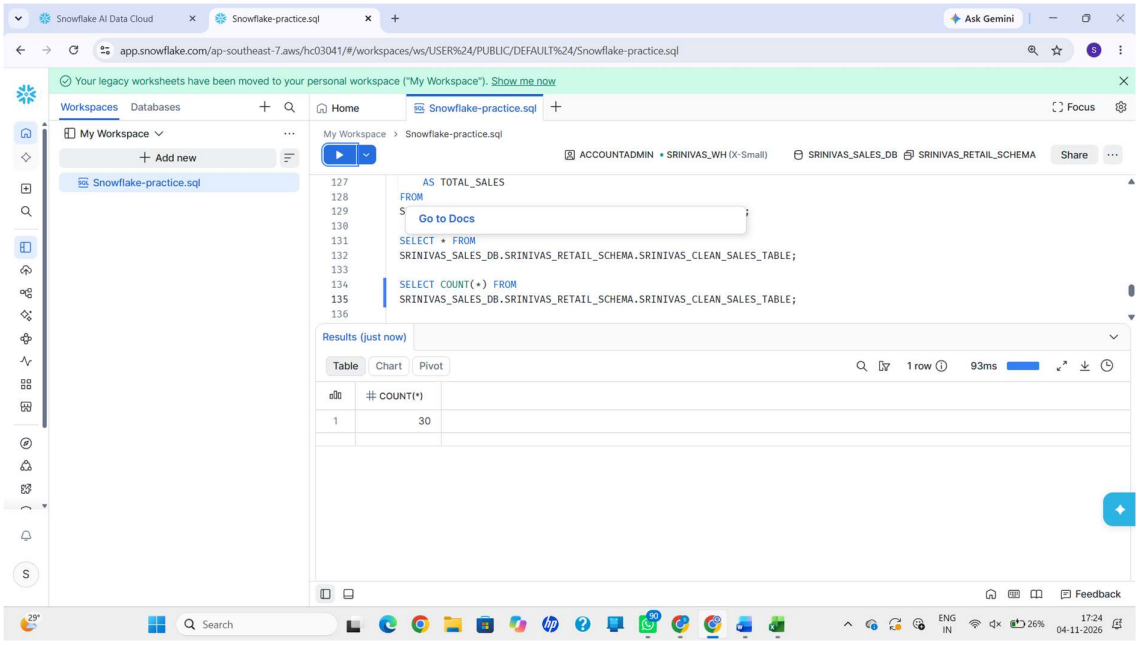


Verify the record count.

```
SELECT COUNT(*) FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE;
```

Expected result: 30 records.

Screenshot 13: Cleaned Data



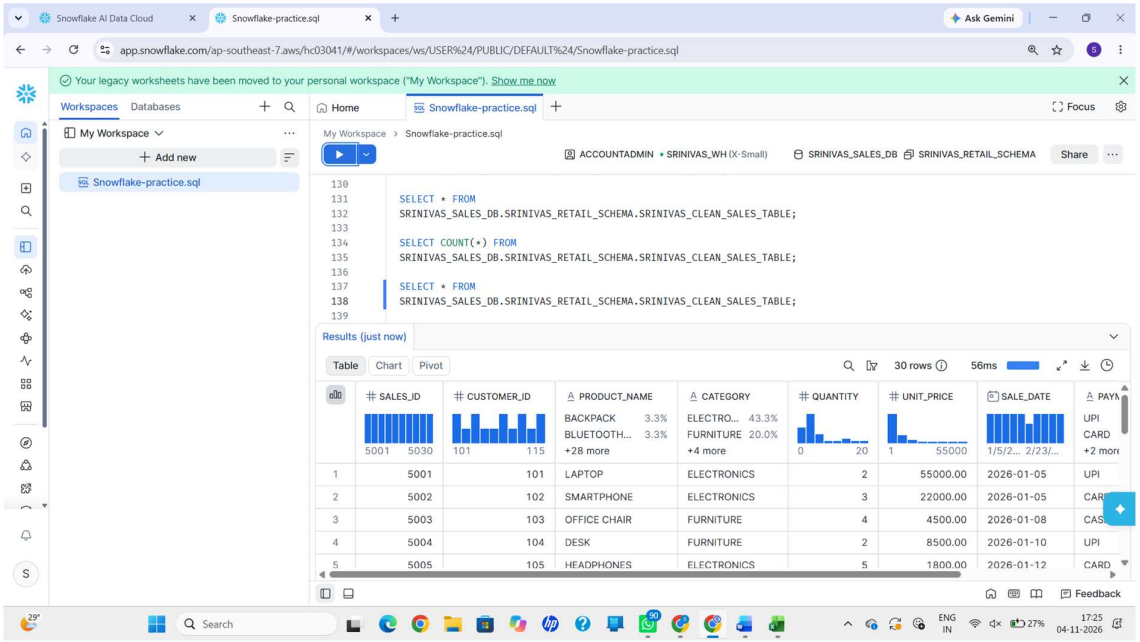
7. SQL QUERY OPERATIONS

STEP 14: DISPLAY ALL ROWS

```
SELECT * FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE;
```

Purpose: Retrieves all columns and records from the clean sales table.

Screenshot 14: Display All Rows

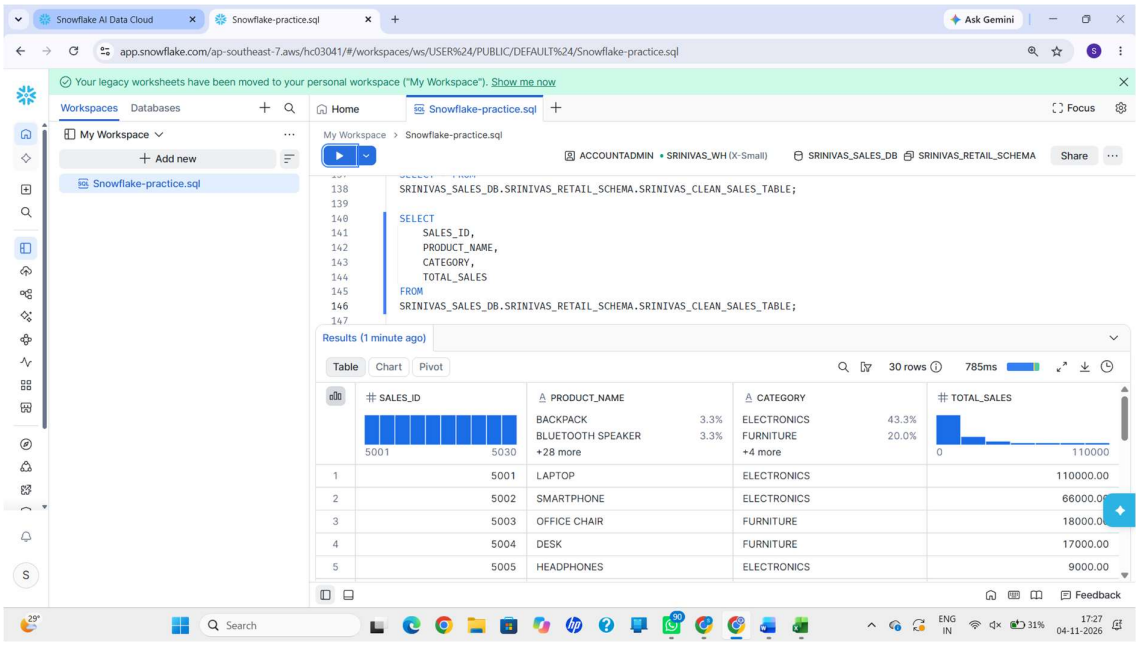


STEP 15: SELECT PARTICULAR COLUMNS

```
SELECT
    SALES_ID,
    PRODUCT_NAME,
    CATEGORY,
    TOTAL_SALES
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE;
```

Purpose: Retrieves only the required columns instead of the entire table.

Screenshot 15: Selected Columns

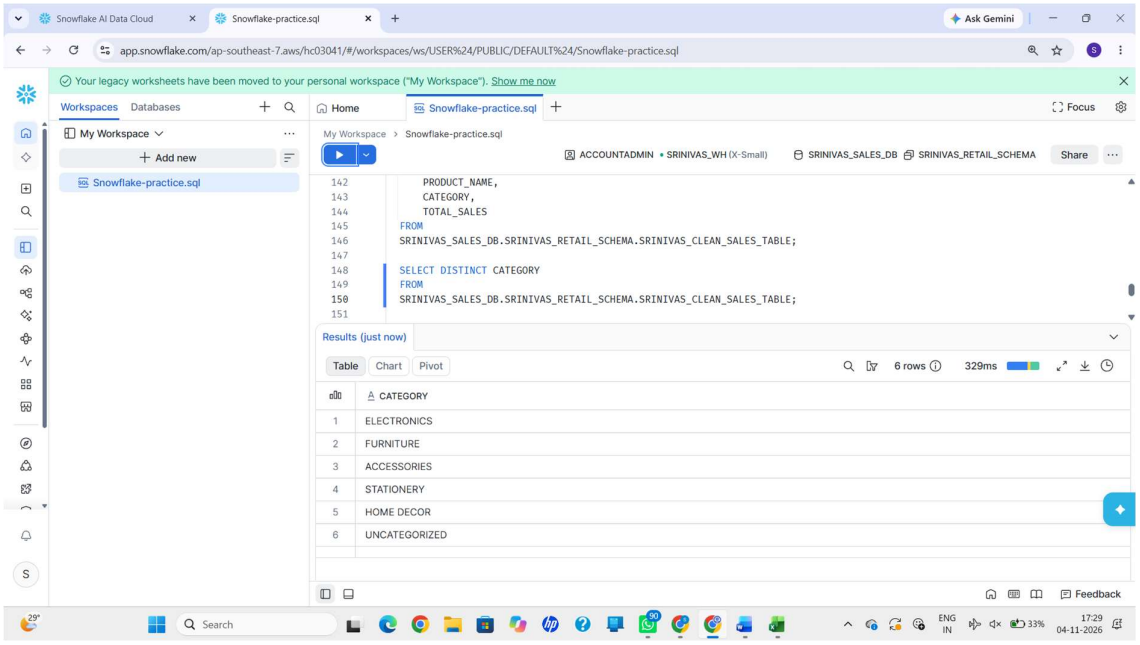


STEP 16: DISPLAY UNIQUE CATEGORIES

```
SELECT DISTINCT CATEGORY
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE;
```

Purpose: Retrieves unique product categories without duplicates.

Screenshot 16: Unique Categories



STEP 17: FILTER RECORDS

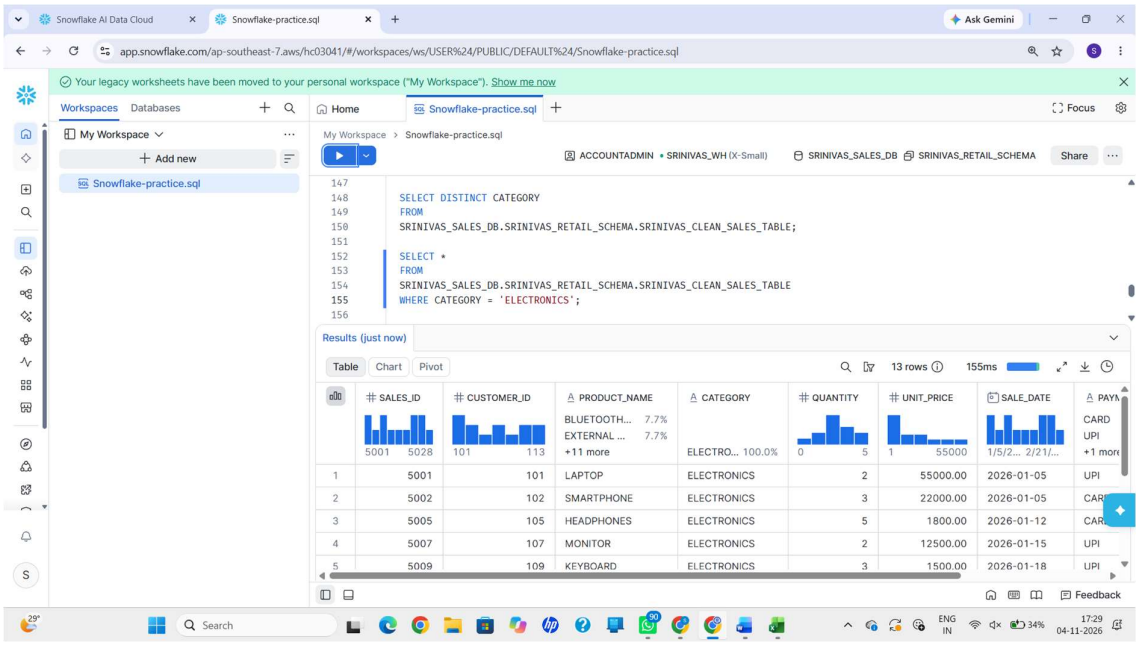
Filter electronics products

```
SELECT *

FROM

SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE

WHERE CATEGORY = 'ELECTRONICS';
```



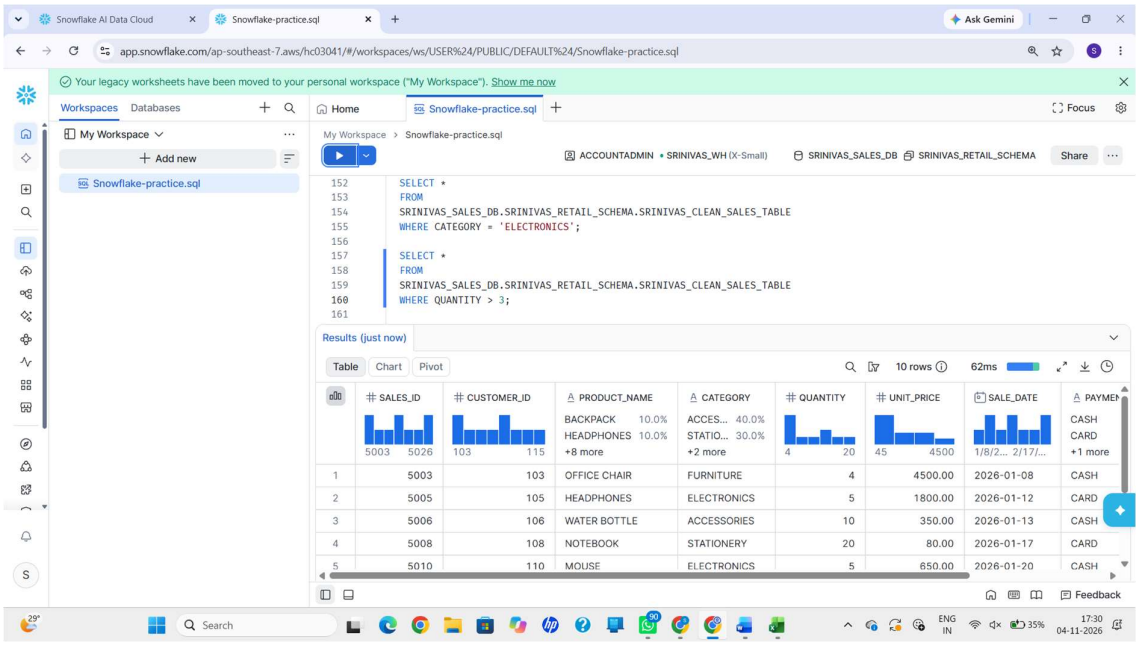
Filter quantity greater than 3

```
SELECT *

FROM

SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TA
BLE

WHERE QUANTITY > 3;
```



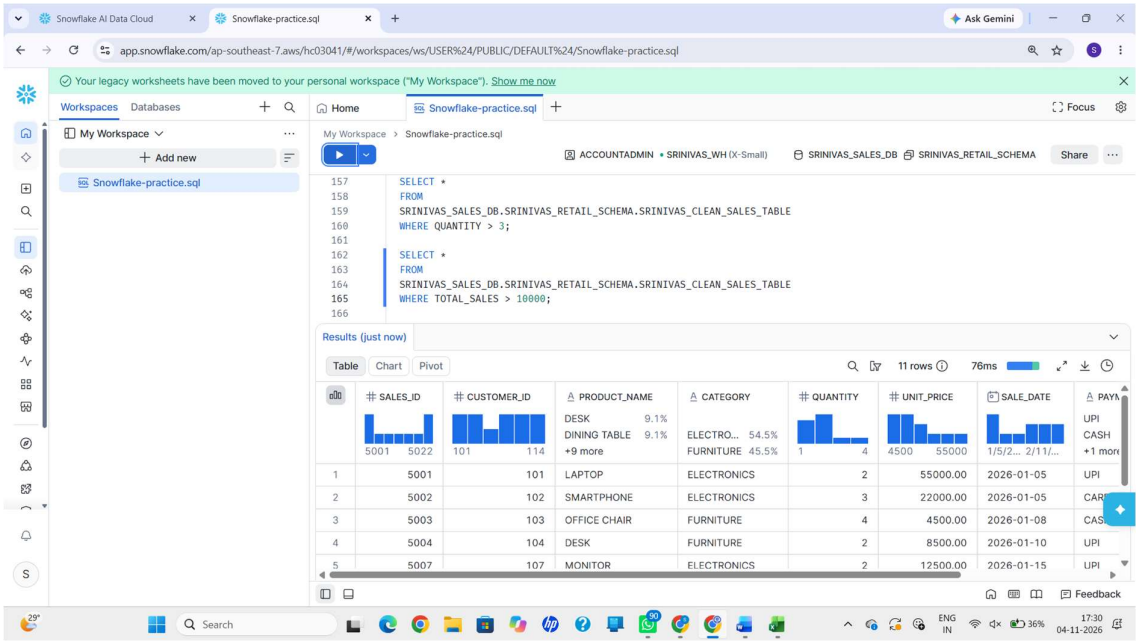
Filter sales greater than ₹10,000

```
SELECT *

FROM

SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TA
BLE

WHERE TOTAL_SALES > 10000;
```



Filter using multiple conditions

SELECT *

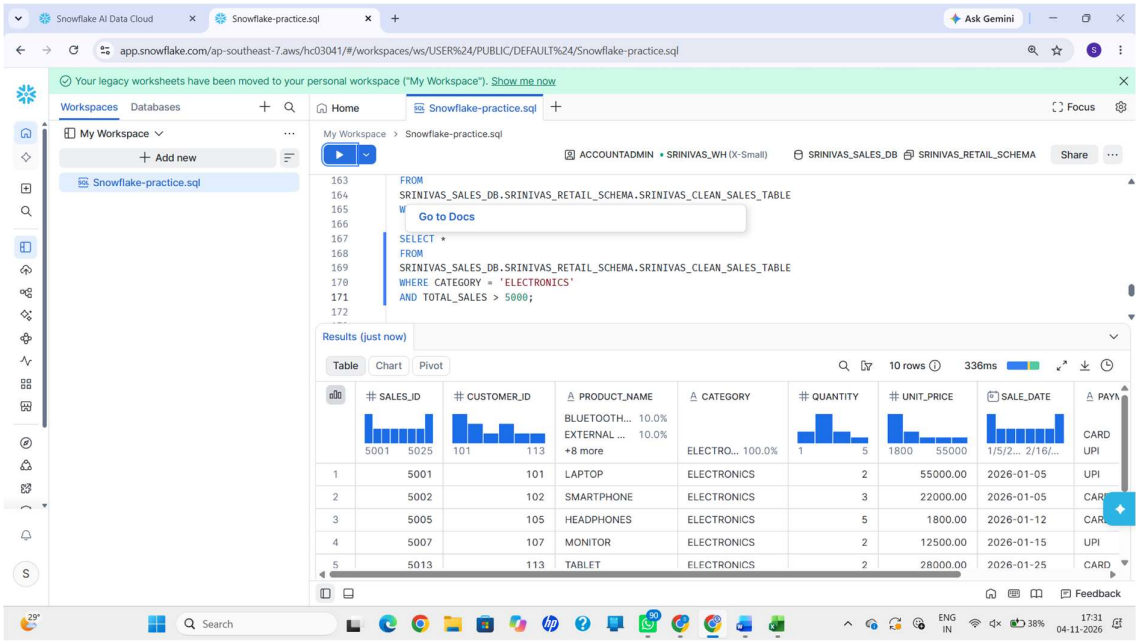
FROM

SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE

WHERE CATEGORY = 'ELECTRONICS'

AND TOTAL_SALES > 5000;

Screenshot 17: Filtering Operations



STEP 18: DATE RANGE FILTERING

Retrieve sales between January 1 and January 31, 2026.

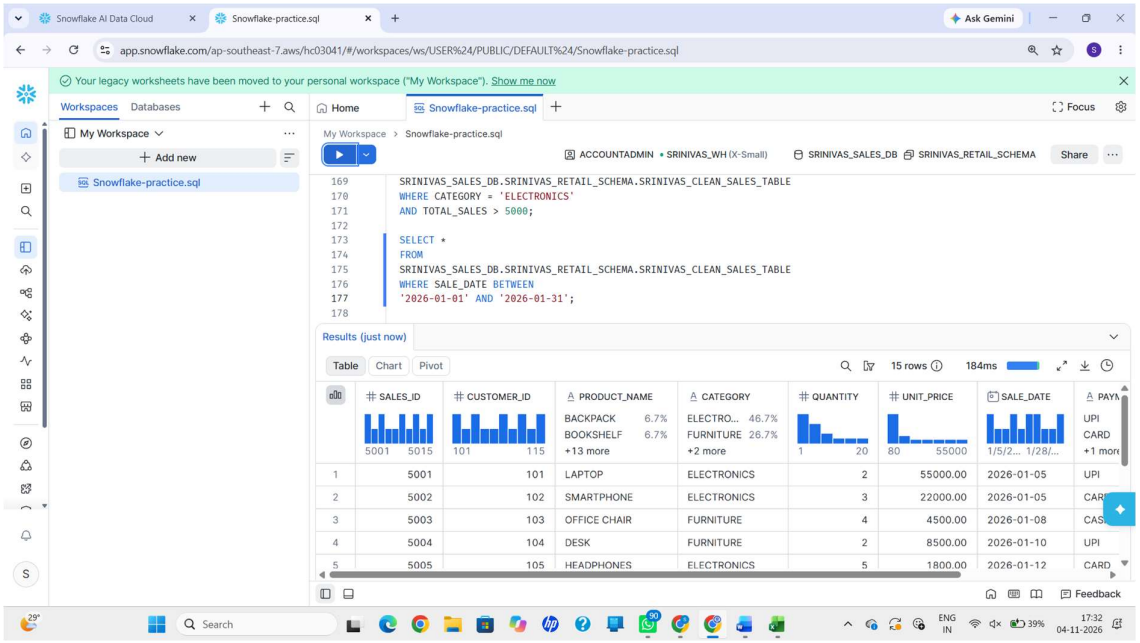
```
SELECT *

FROM

SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE

WHERE SALE_DATE BETWEEN

'2026-01-01' AND '2026-01-31';
```



Retrieve February sales.

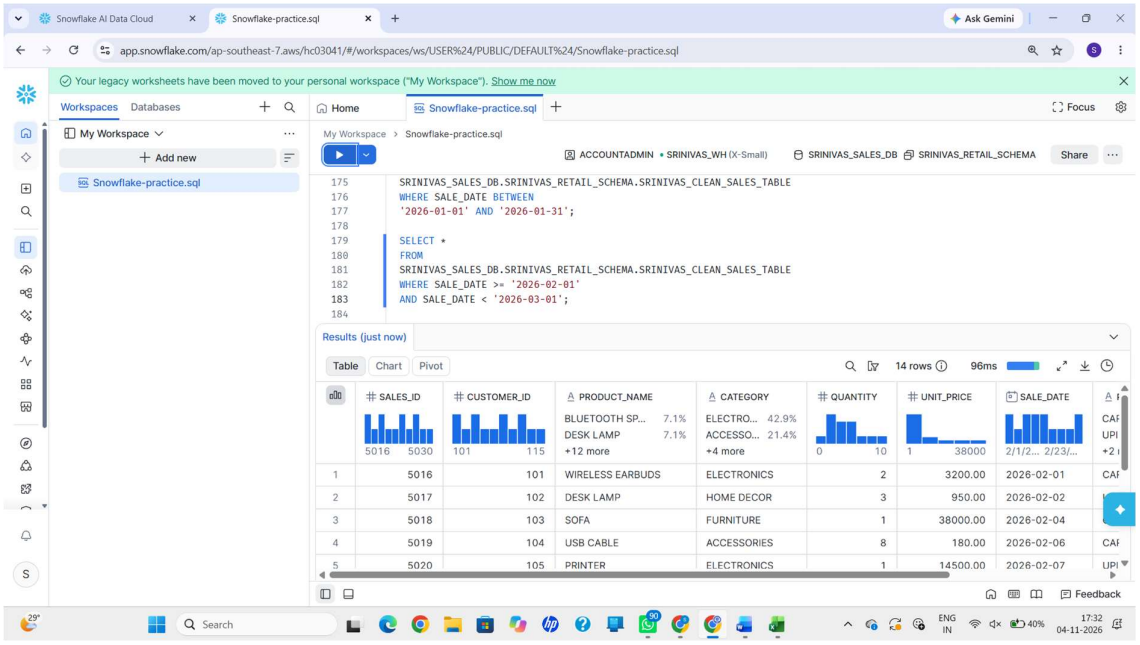
```
SELECT *

FROM

SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE

WHERE SALE_DATE >= '2026-02-01'

AND SALE_DATE < '2026-03-01';
```



Find records with missing dates.

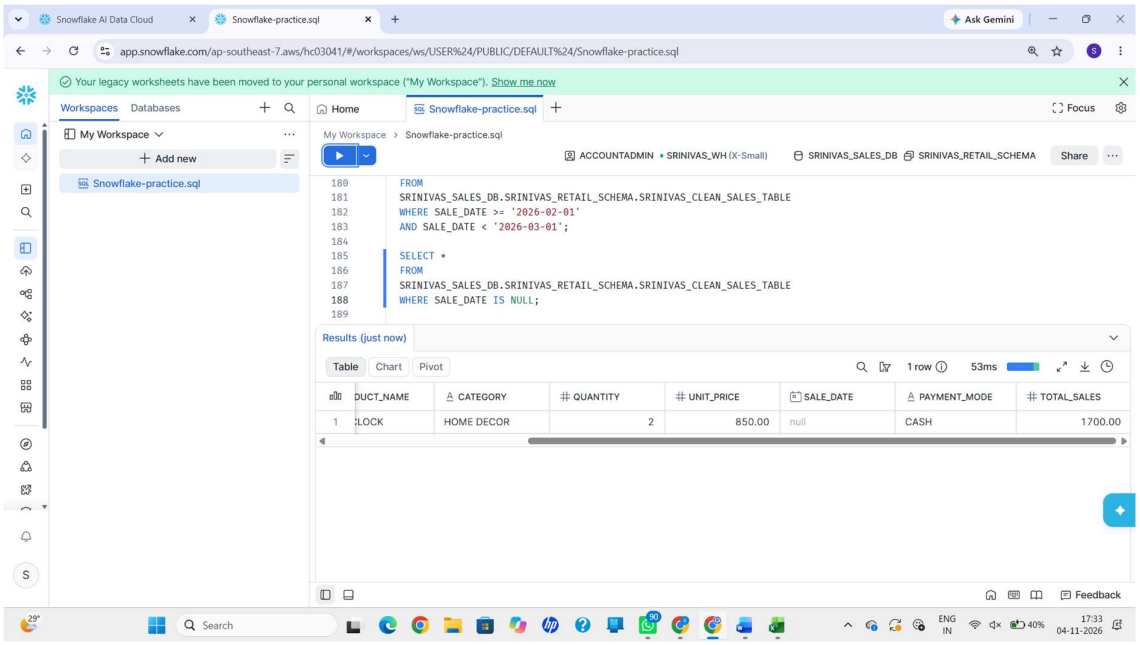
```
SELECT *

FROM

SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE

WHERE SALE_DATE IS NULL;
```

Screenshot 18: Date Filtering



8. AGGREGATION OPERATIONS

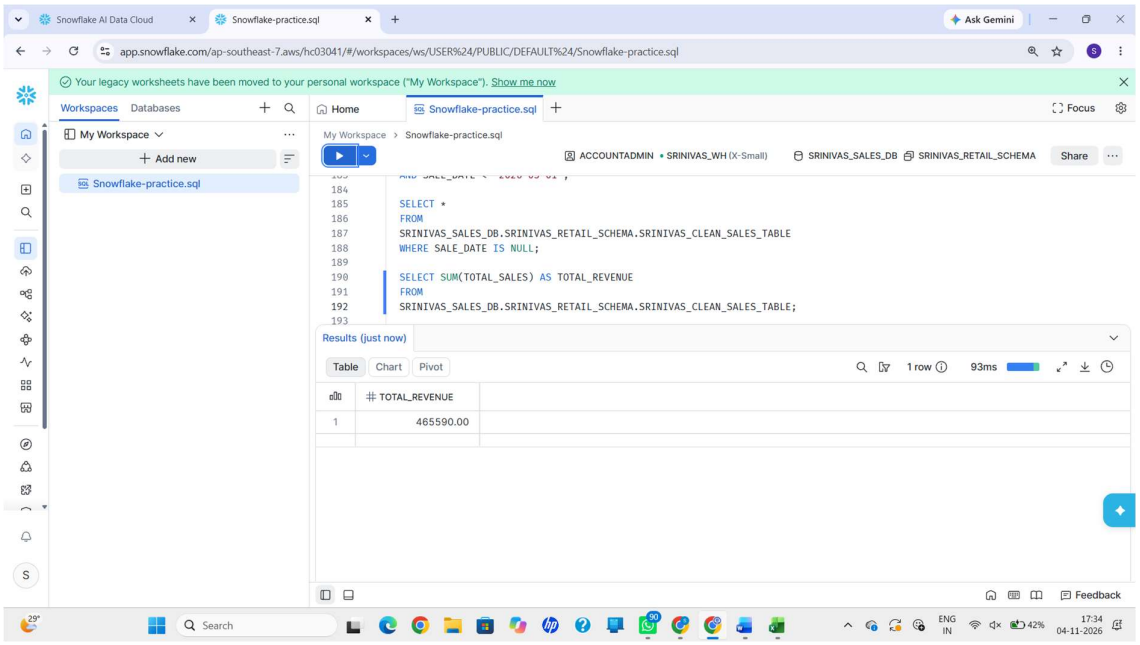
STEP 19: TOTAL SALES

```
SELECT SUM(TOTAL_SALES) AS TOTAL_REVENUE

FROM

SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE;
```

Purpose: Calculates the total revenue generated from all sales transactions.



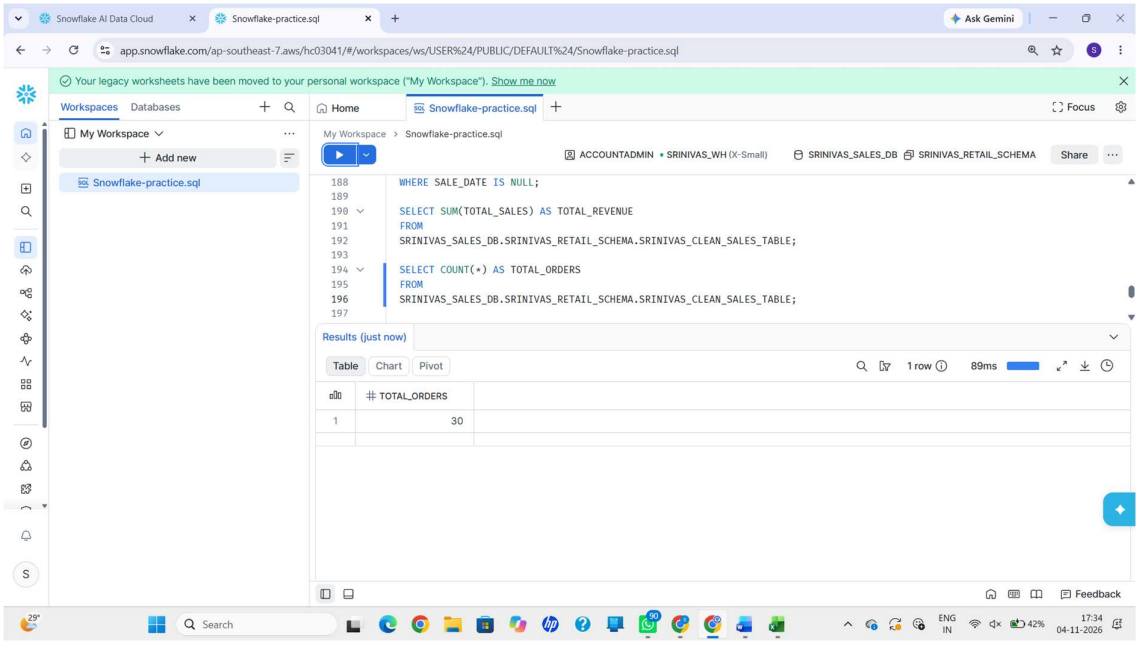
STEP 20: COUNT NUMBER OF RECORDS

```
SELECT COUNT(*) AS TOTAL_ORDERS

FROM

SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE;
```

Purpose: Counts the total number of sales records.



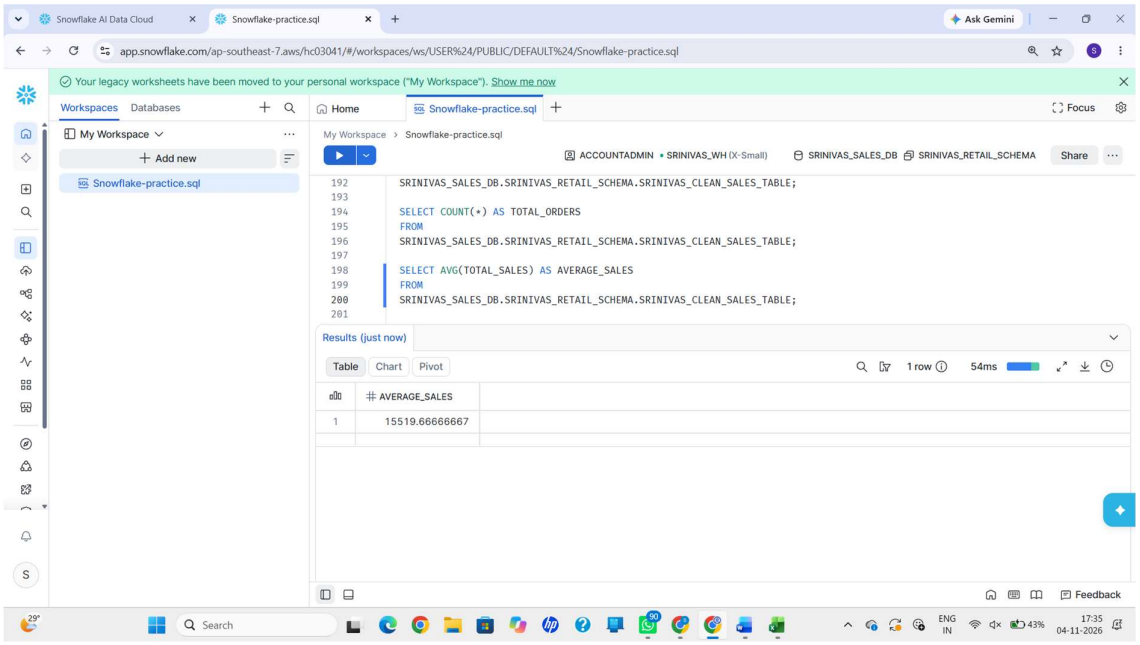
STEP 21: AVERAGE SALES

```
SELECT AVG(TOTAL_SALES) AS AVERAGE_SALES

FROM

SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE;
```

Purpose: Calculates the average sales value.

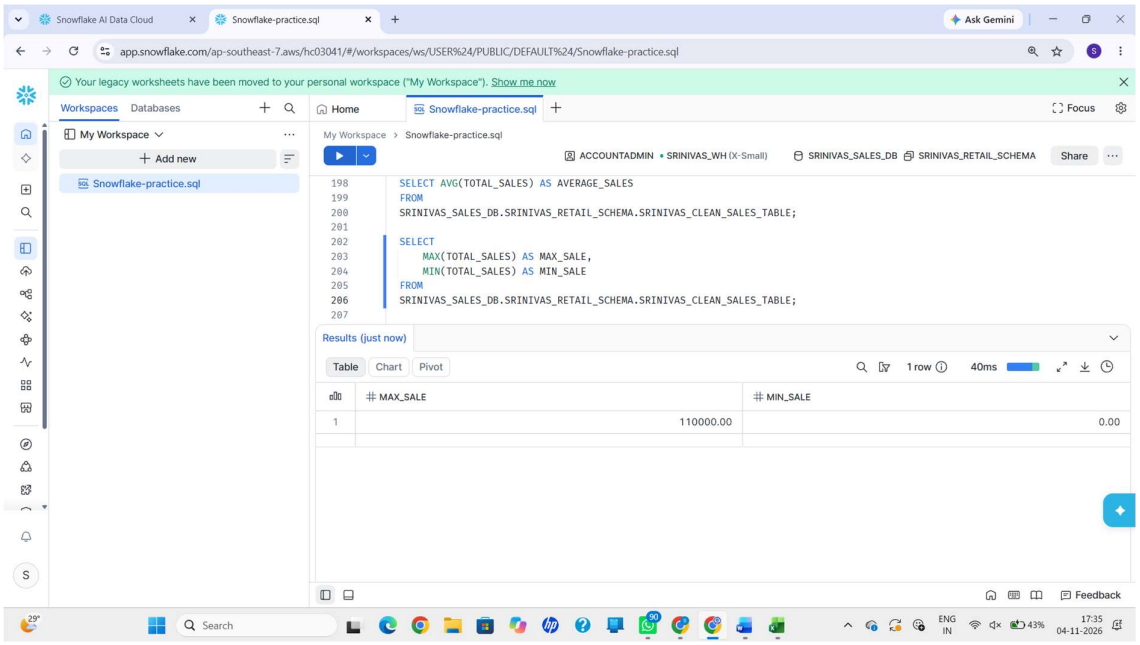


STEP 22: MAXIMUM AND MINIMUM SALES

```
SELECT
```

```
    MAX(TOTAL_SALES) AS MAX_SALE,
    MIN(TOTAL_SALES) AS MIN_SALE
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE;
```

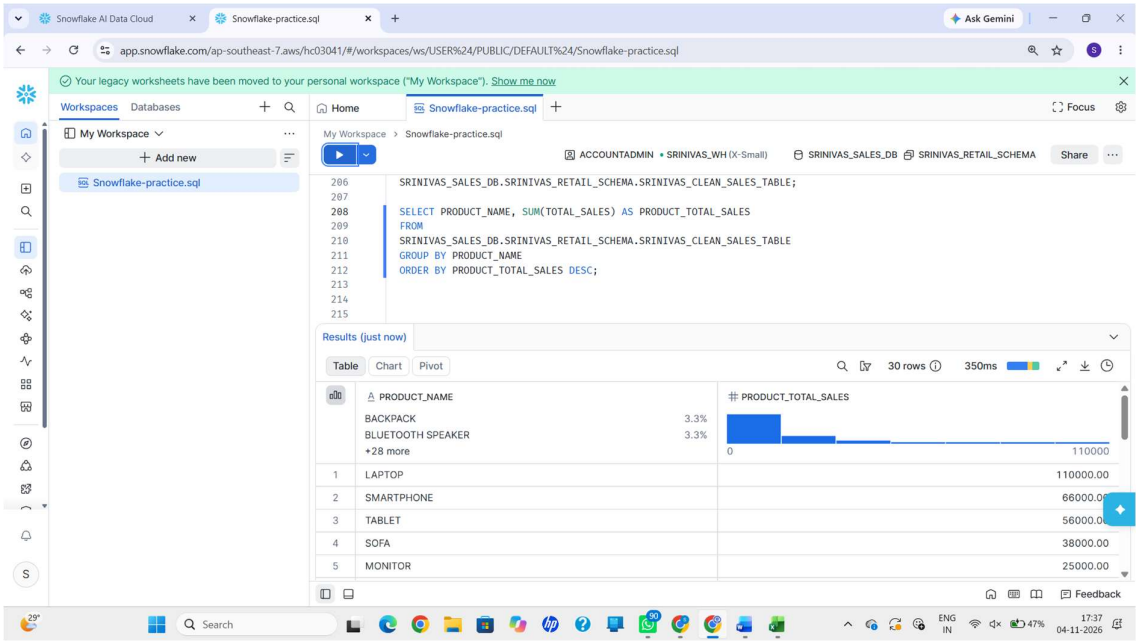
Purpose: Identifies the highest and lowest transaction values.



STEP 23: PRODUCT-WISE SALES

```
SELECT PRODUCT_NAME, SUM(TOTAL_SALES) AS PRODUCT_TOTAL_SALES
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE
GROUP BY PRODUCT_NAME
ORDER BY PRODUCT_TOTAL_SALES DESC;
```

Purpose: Calculates the total revenue generated by each product.



STEP 24: CATEGORY-WISE SALES

```
SELECT   CATEGORY, SUM(TOTAL_SALES) AS CATEGORY_REVENUE

FROM

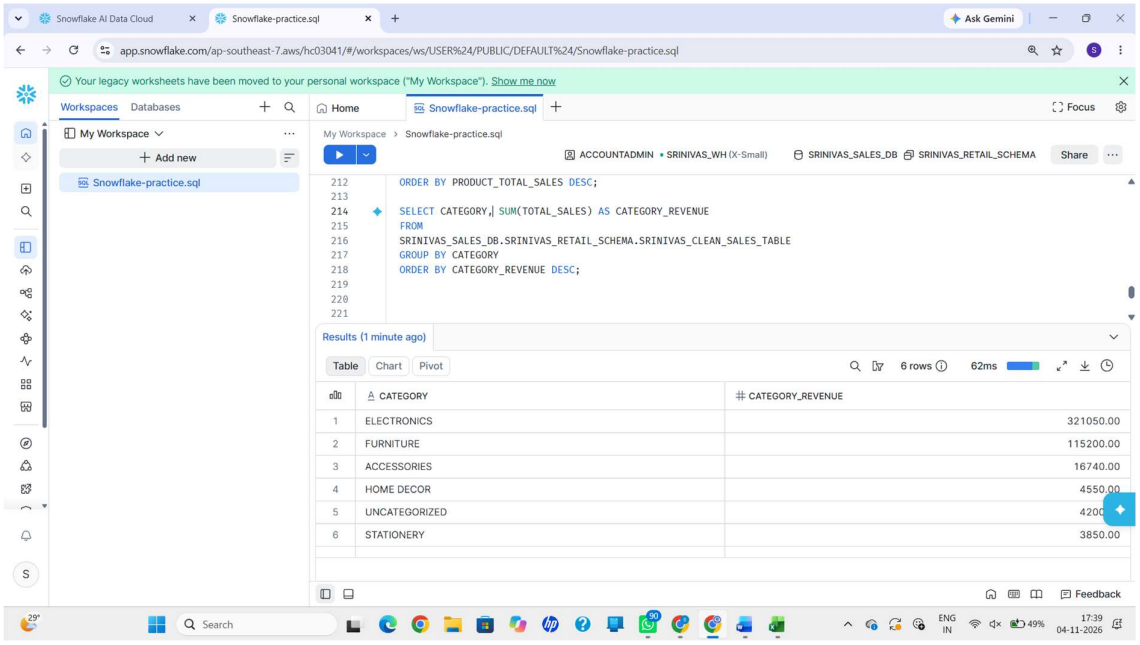
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE

GROUP BY CATEGORY

ORDER BY CATEGORY_REVENUE DESC;
```

Purpose: Compares revenue across different product categories.

Screenshot 19: Aggregation and Grouping Operations



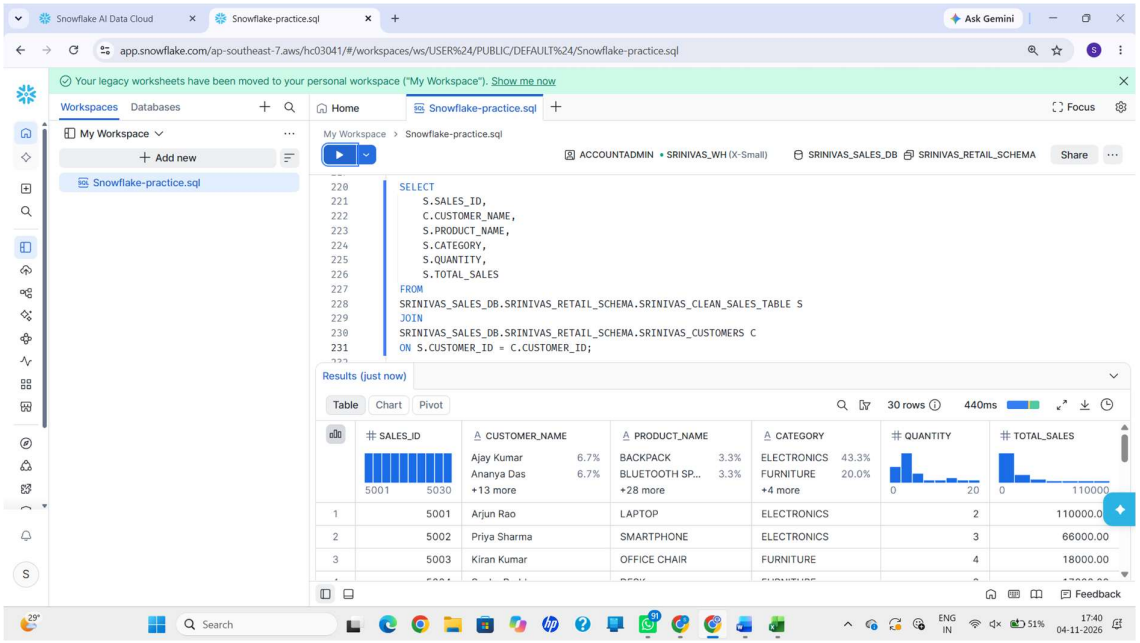
9. JOIN OPERATIONS

STEP 25: JOIN CUSTOMERS AND SALES

Combine customer details with sales transaction information.

```
SELECT
    S.SALES_ID,
    C.CUSTOMER_NAME,
    S.PRODUCT_NAME,
    S.CATEGORY,
    S.QUANTITY,
    S.TOTAL_SALES
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE S
JOIN
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CUSTOMERS C
ON S.CUSTOMER_ID = C.CUSTOMER_ID;
```

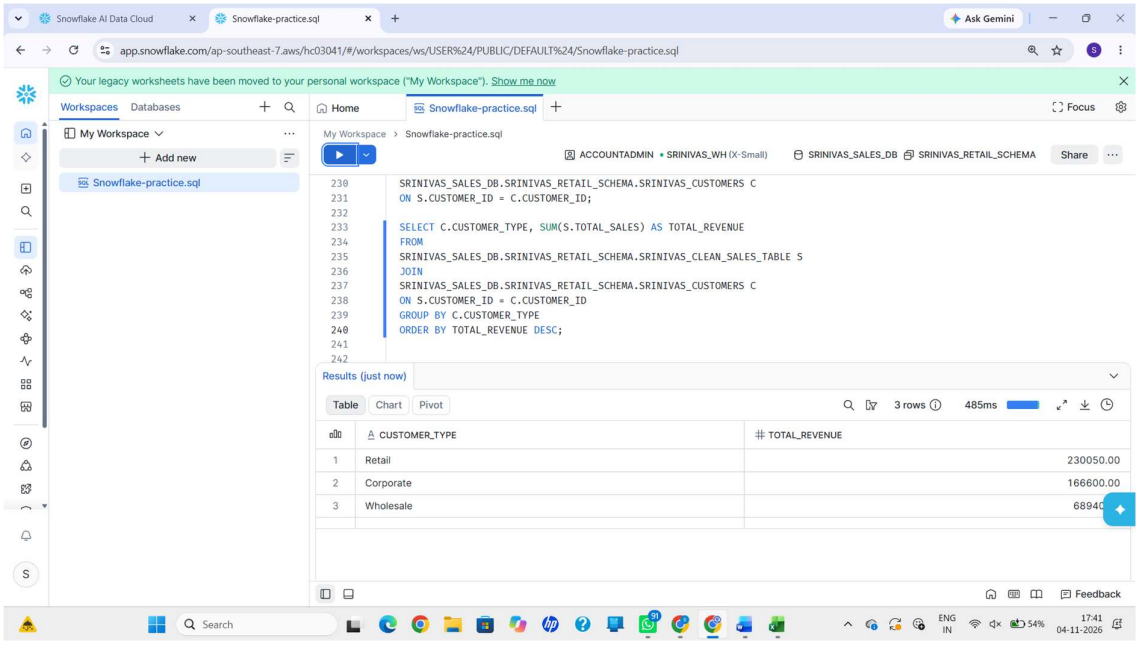
Purpose: Combines two tables using the common CUSTOMER_ID column.



STEP 26: TOTAL SALES BY CUSTOMER TYPE

```
SELECT C.CUSTOMER_TYPE, SUM(S.TOTAL_SALES) AS TOTAL_REVENUE
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE S
JOIN
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CUSTOMERS C
ON S.CUSTOMER_ID = C.CUSTOMER_ID
GROUP BY C.CUSTOMER_TYPE
ORDER BY TOTAL_REVENUE DESC;
```

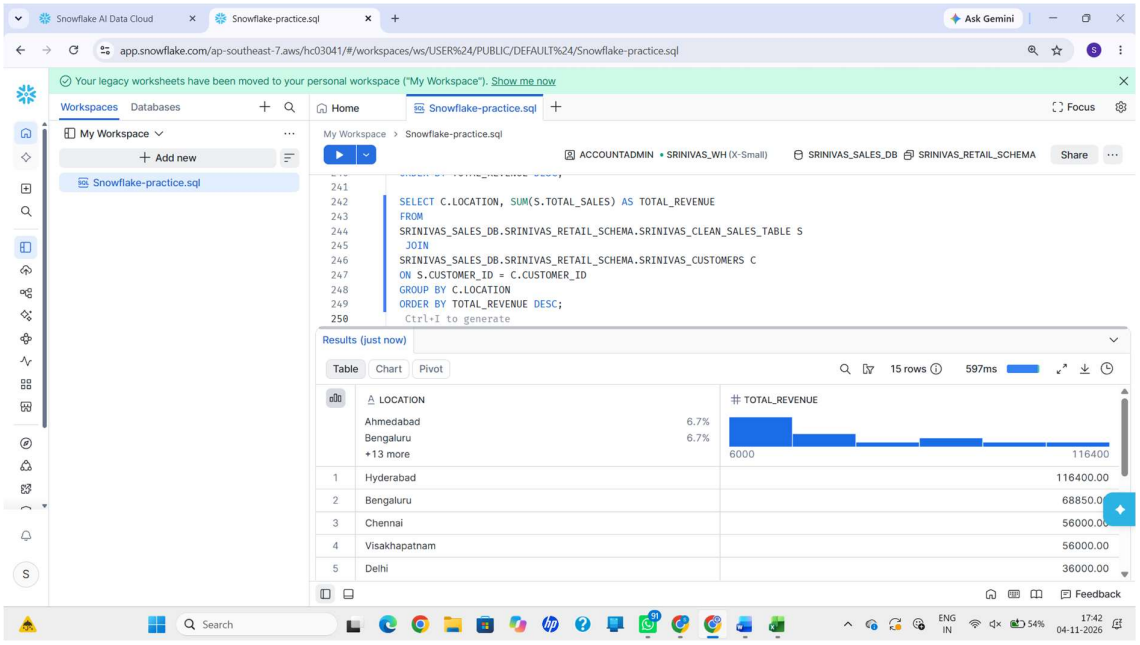
Purpose: Calculates revenue generated by Retail, Corporate and Wholesale customers.



STEP 27: LOCATION-WISE SALES

```
SELECT C.LOCATION, SUM(S.TOTAL_SALES) AS TOTAL_REVENUE
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE S
 JOIN
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CUSTOMERS C
ON S.CUSTOMER_ID = C.CUSTOMER_ID
GROUP BY C.LOCATION
ORDER BY TOTAL_REVENUE DESC;
```

Purpose: Identifies the revenue generated from each customer location.



STEP 28: CUSTOMER-WISE TOTAL SALES

```
SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(S.TOTAL_SALES) AS
TOTAL_REVENUE

FROM

SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE S

JOIN

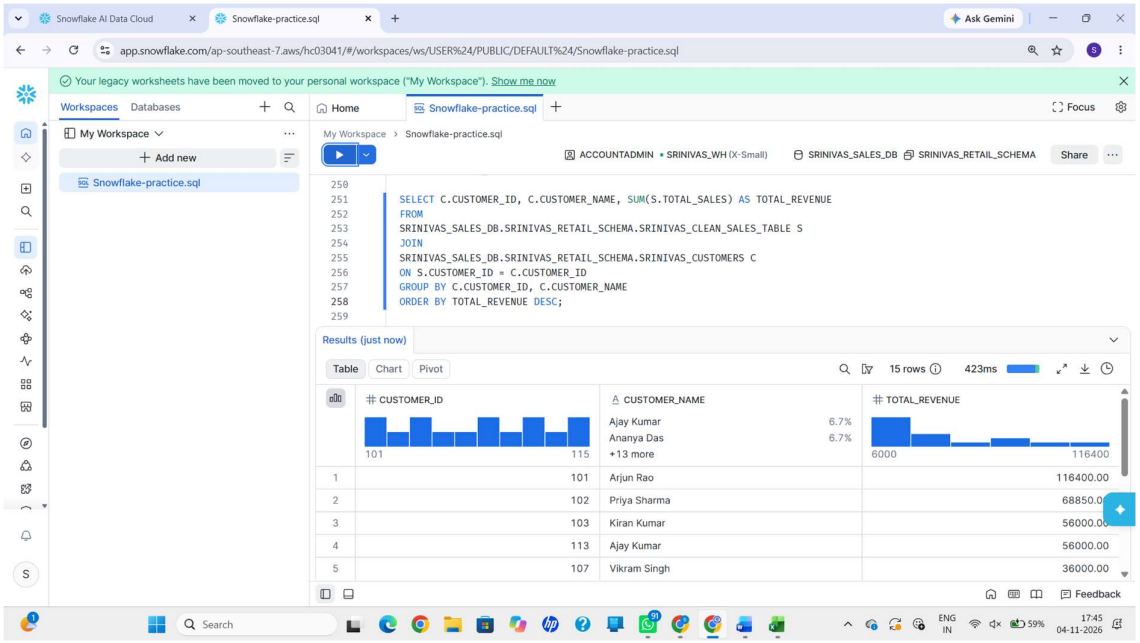
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CUSTOMERS C

ON S.CUSTOMER_ID = C.CUSTOMER_ID

GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME

ORDER BY TOTAL_REVENUE DESC;
```

Purpose: Calculates the total purchase amount for each customer.

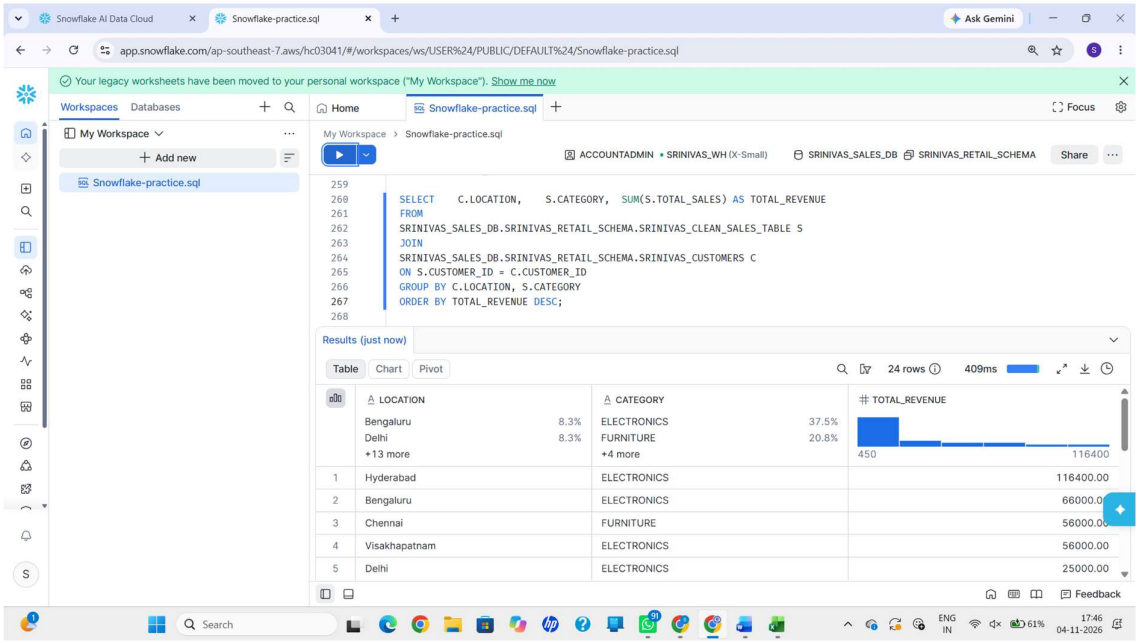


STEP 29: CATEGORY-WISE SALES BY LOCATION

```
SELECT C.LOCATION, S.CATEGORY, SUM(S.TOTAL_SALES) AS TOTAL_REVENUE
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TABLE S
JOIN
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CUSTOMERS C
ON S.CUSTOMER_ID = C.CUSTOMER_ID
GROUP BY C.LOCATION, S.CATEGORY
ORDER BY TOTAL_REVENUE DESC;
```

Purpose: Analyses the performance of product categories across different locations.

Screenshot 20: JOIN and Customer-Based Analysis



---

10. LARGE DATASET GENERATION

STEP 30: CREATE TABLE WITH 1 LAKH RECORDS

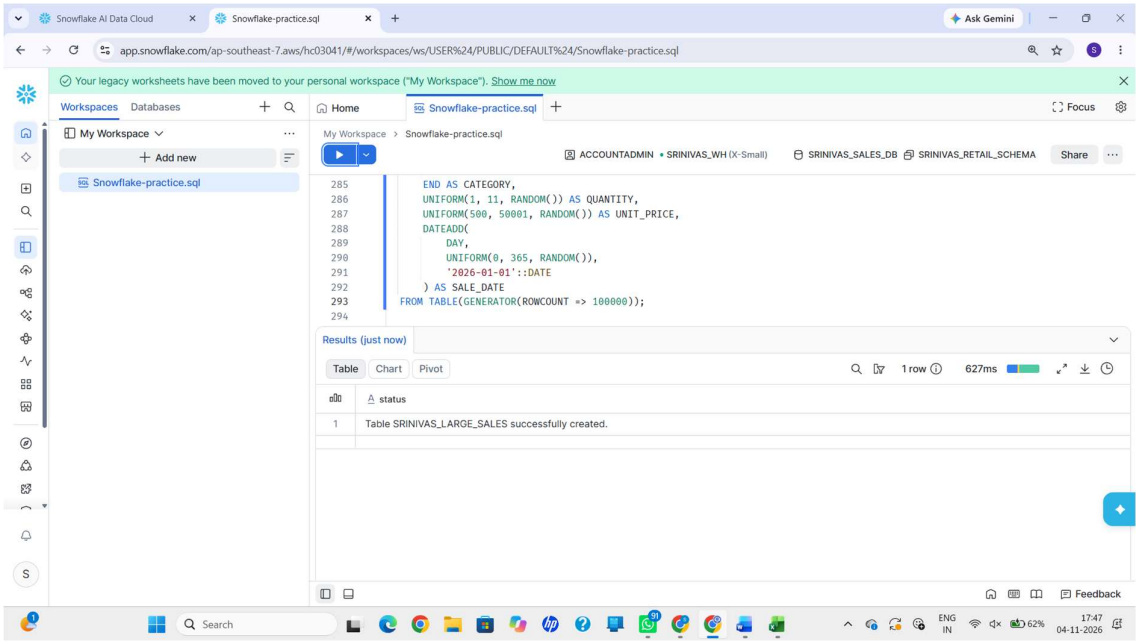
Use Snowflake's GENERATOR function to generate 100,000 sample sales records.

```
CREATE OR REPLACE TABLE
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_LARGE_SALES AS
SELECT
  ROW_NUMBER() OVER (ORDER BY SEQ4()) AS SALES_ID,
  UNIFORM(101, 116, RANDOM()) AS CUSTOMER_ID,
  CASE MOD(SEQ4(), 5)
    WHEN 0 THEN 'LAPTOP'
    WHEN 1 THEN 'MOBILE'
    WHEN 2 THEN 'KEYBOARD'
    WHEN 3 THEN 'MONITOR'
    ELSE 'MOUSE'
  END AS PRODUCT_NAME,
  CASE MOD(SEQ4(), 3)
    WHEN 0 THEN 'ELECTRONICS'
    WHEN 1 THEN 'ACCESSORIES'
```

```
    ELSE 'FURNITURE'
  END AS CATEGORY,
  UNIFORM(1, 11, RANDOM()) AS QUANTITY,
  UNIFORM(500, 50001, RANDOM()) AS UNIT_PRICE,
  DATEADD(
    DAY,
    UNIFORM(0, 365, RANDOM()),
    '2026-01-01'::DATE
  ) AS SALE_DATE
FROM TABLE(GENERATOR(ROWCOUNT => 100000));
```

Explanation:

- GENERATOR(): Generates a specified number of rows.
- ROWCOUNT => 100000: Generates 100,000 records.
- SEQ4(): Generates a sequence of integers.
- RANDOM(): Generates random values.
- UNIFORM(): Generates random values within a specified range.
- CASE: Assigns product names and categories.
- DATEADD(): Generates different transaction dates.

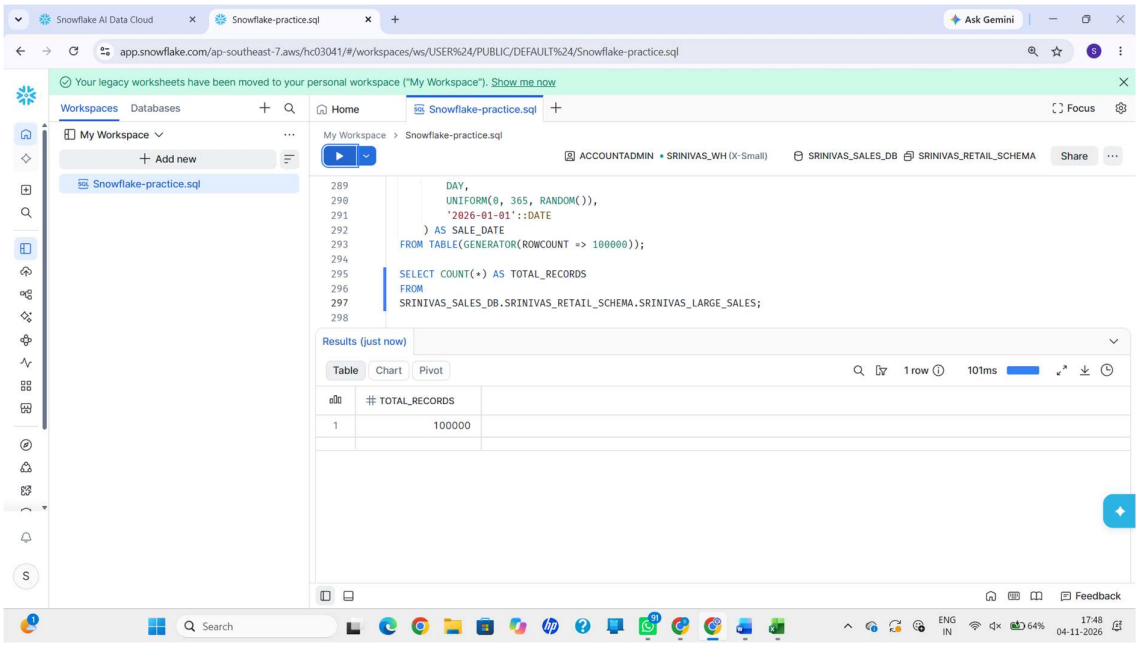


STEP 31: VERIFY RECORD COUNT

```
SELECT COUNT(*) AS TOTAL_RECORDS
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_LARGE_SALES;
```

Expected output: 100000.

Screenshot 21: Table Containing 1 Lakh Records



11. SQL OPTIMIZATION

STEP 32: AVOID SELECT *

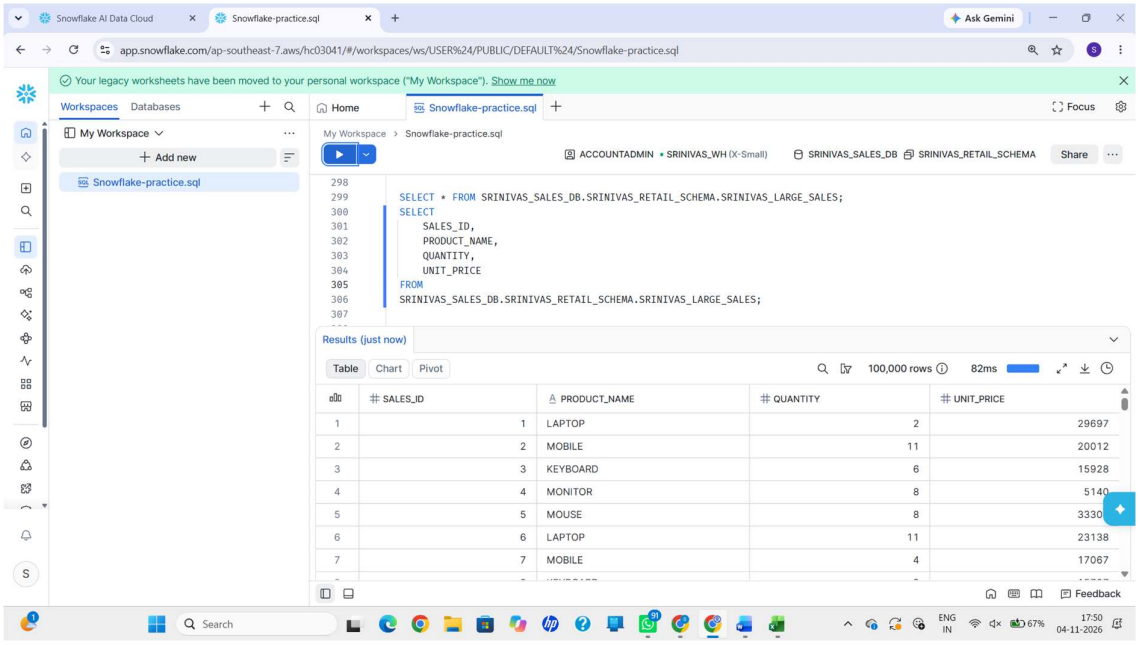
Instead of retrieving all columns:

```
SELECT *
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_LARGE_SALES;
```

Retrieve only the necessary columns:

```
SELECT
   SALES_ID,
   PRODUCT_NAME,
   QUANTITY,
   UNIT_PRICE
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_LARGE_SALES;
```

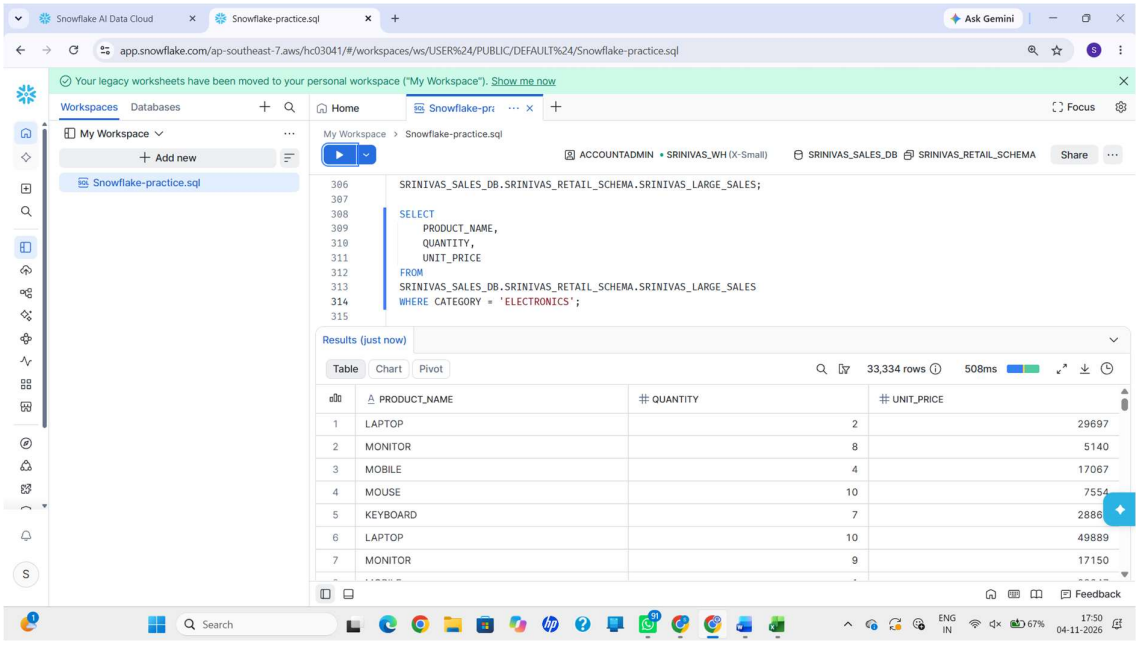
Purpose: Reduces unnecessary data retrieval.



STEP 33: USE FILTERING

```
SELECT
   PRODUCT_NAME,
   QUANTITY,
   UNIT_PRICE
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_LARGE_SALES
WHERE CATEGORY = 'ELECTRONICS';
```

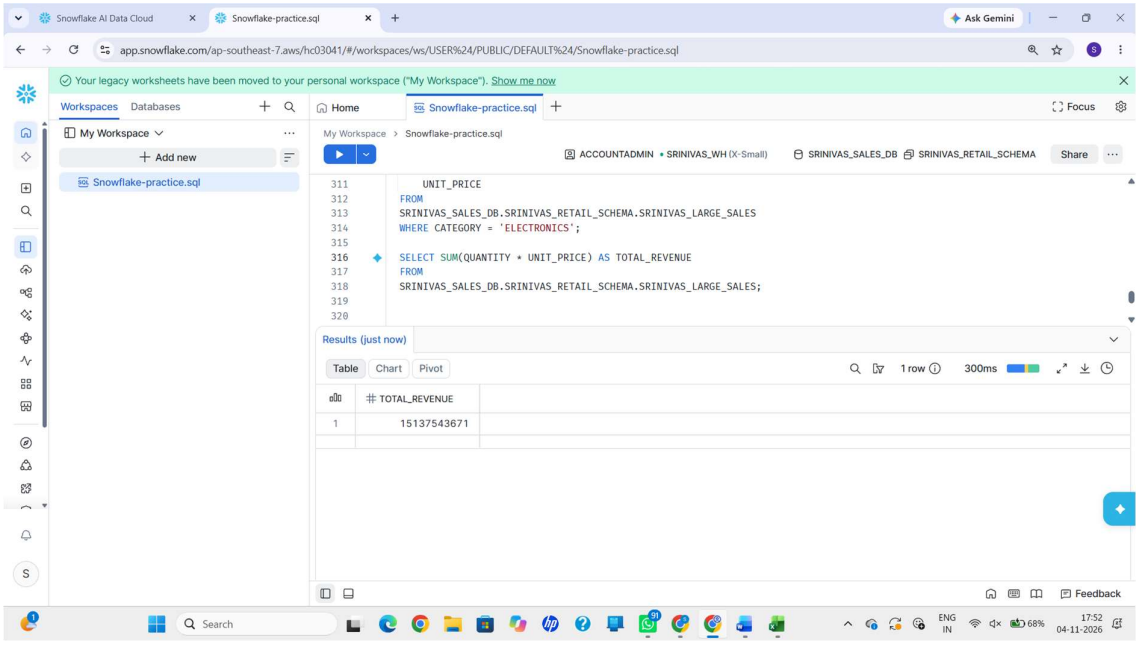
Purpose: Retrieves only the records required for analysis.



STEP 34: USE AGGREGATION

```
SELECT  SUM(QUANTITY * UNIT_PRICE) AS TOTAL_REVENUE
FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_LARGE_SALES;
```

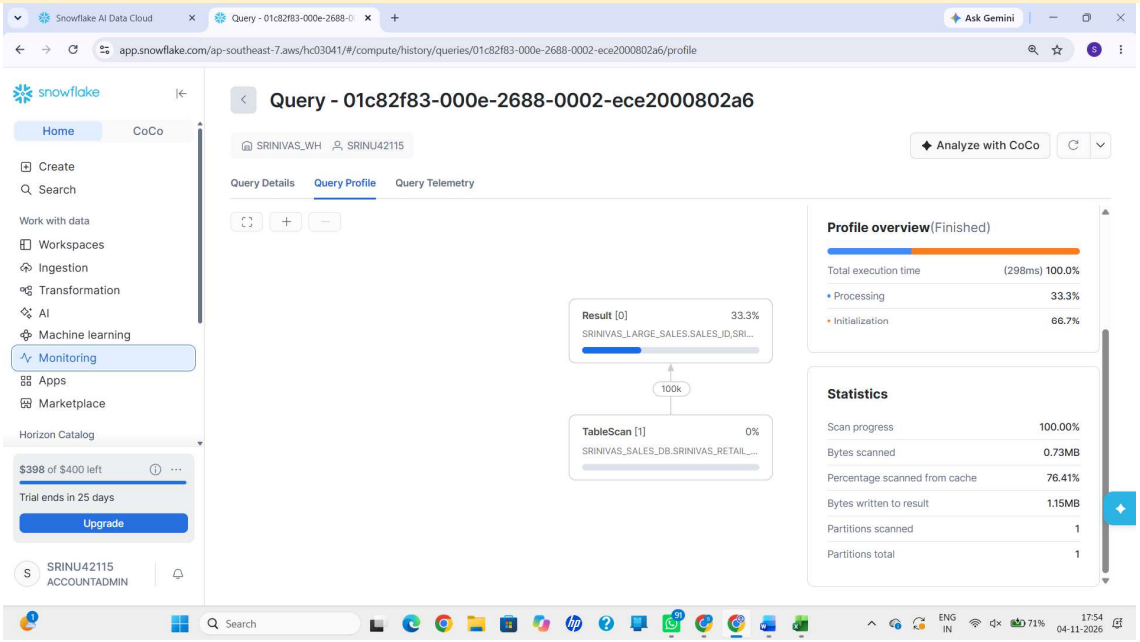
Purpose: Calculates the required result directly without retrieving all individual records.



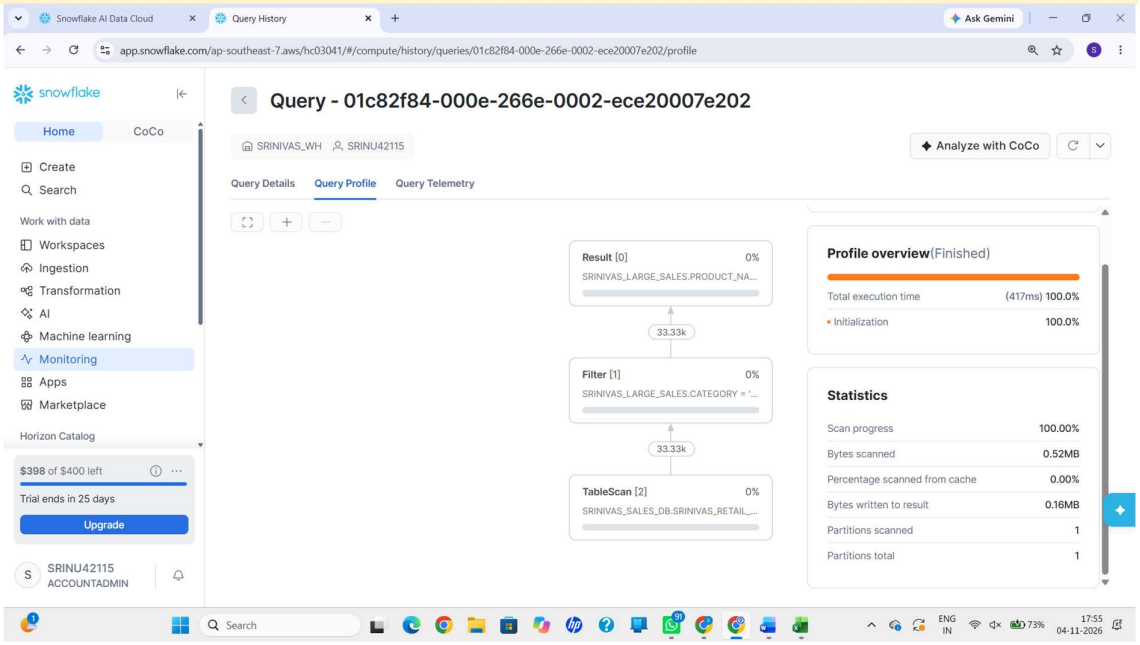
STEP 35: QUERY PROFILE

Screenshot 22: Query Profile

```
SELECT * FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_LARGE_SALES;
```




```
SELECT PRODUCT_NAME, QUANTITY,  UNIT_PRICE FROM
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_LARGE_SALES
WHERE CATEGORY = 'ELECTRONICS';
```



STEP 36: DROP QUERY

```
DROP TABLE IF EXISTS
```

```
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CUSTOMERS;


DROP TABLE IF EXISTS
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_SALES_RAW;


DROP TABLE IF EXISTS
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CLEAN_SALES_TA
BLE;


DROP TABLE IF EXISTS
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_LARGE_SALES;


DROP STAGE IF EXISTS
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_SALES_STAGE;


DROP FILE FORMAT IF EXISTS
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CSV_FORMAT;

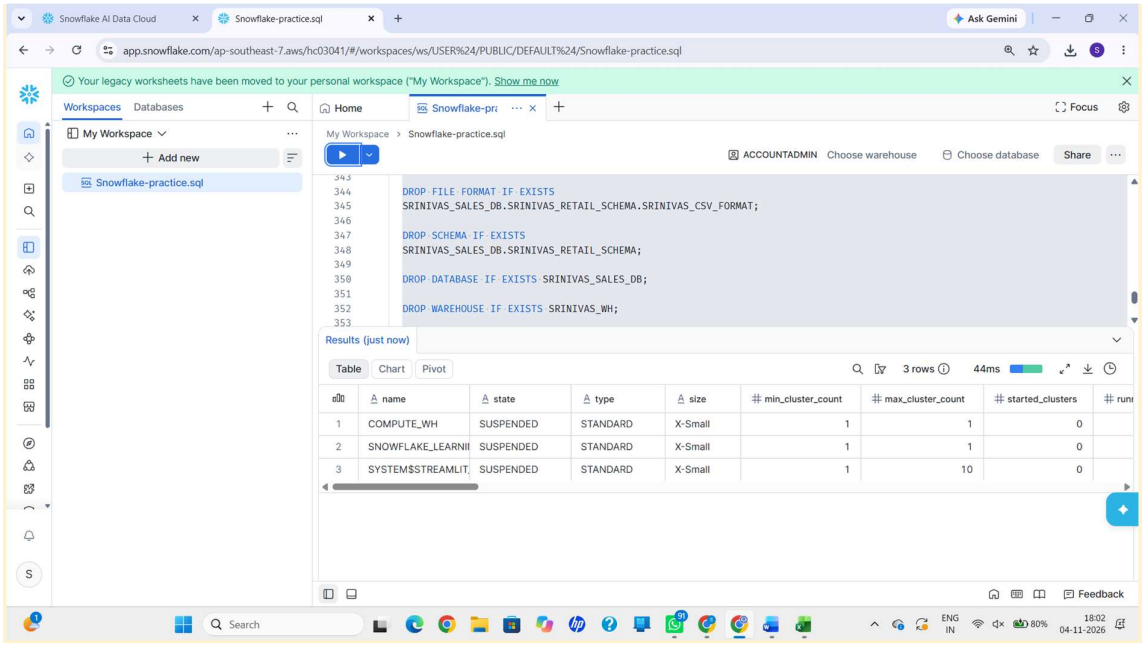

DROP SCHEMA IF EXISTS
SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA;


DROP DATABASE IF EXISTS SRINIVAS_SALES_DB;


DROP WAREHOUSE IF EXISTS SRINIVAS_WH;


SHOW DATABASES;


SHOW WAREHOUSES;
```


Snowflake AI Data Cloud
Snowflake-practice.sql
Ask Gemini
app.snowflake.com/ap-southeast-7.aws/hc03041/#/workspaces/ws/USER%24/PUBLIC/DEFAULT%24/Snowflake-practice.sql
Your legacy worksheets have been moved to your personal workspace ("My Workspace"). Show me now
Workspaces
Databases
My Workspace
Add new
Snowflake-practice.sql
Home
Snowflake-pra
Focus
My Workspace › Snowflake-practice.sql
ACCOUNTADMIN
Choose warehouse
Choose database
Share
343
344 DROP FILE FORMAT IF EXISTS
345 SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA.SRINIVAS_CSV_FORMAT;
346
347 DROP SCHEMA IF EXISTS
348 SRINIVAS_SALES_DB.SRINIVAS_RETAIL_SCHEMA;
349
350 DROP DATABASE IF EXISTS SRINIVAS_SALES_DB;
351
352 DROP WAREHOUSE IF EXISTS SRINIVAS_WH;
353
Results (just now)
Table
Chart
Pivot
3 rows
44ms
name
state
type
size
min_cluster_count
max_cluster_count
started_clusters
1 COMPUTE_WH SUSPENDED STANDARD X-Small 1 1 0
2 SNOWFLAKE_LEARNII SUSPENDED STANDARD X-Small 1 1 0
3 SYSTEM$STREAMLIT SUSPENDED STANDARD X-Small 1 10 0
Feedback
Search
ENG
IN
80%
18:02
04-11-2026